

PeliHylly

Paperinen työpaketti

Oma pelikirjastopalvelu: React, Express, SQLite ja julkaisu tuotantoon

Työviikot 1–18 · luovutus työviikon 18 lopussa

Tämä paketti on aikataulu ja tarkistuslista tilanteisiin, joissa sivusto ei ole auki. Projektipäiväkirja kirjoitetaan sivustolla ja viedään repositoryn project-docs-kansioon. Rasti tässä vihossa ei ole palautus: työ on aina Git-repositoryssa.

Julkiseen repositoryyn ei laiteta henkilötietoja, salasanoja eikä ympäristömuuttujien arvoja.

Viikot ovat työviikkoja opiskelijan omasta aloituksesta; paketissa ei ole kalenteripäivämääriä.

Rasti tässä vihossa ei ole palautus: työnäyte on aina Git-repositoryssa.

Mitä rakennat ja miten etenet

PeliHylly on Pelikellari ry:n jäsenten verkkopalvelu. Jokainen kirjaa palveluun pelinsä laitteineen, tähtiarvioineen ja kommentteineen ja vaihtaa pelin tilaa yhdellä napilla. Julkinen profiili näyttää kenelle tahansa, mitä jäsen pelaa nyt ja miten hänen hyllynsä on muuttunut.

1. Julkinen profiili näyttää, mitä jäsen pelaa nyt ja montako peliä on missäkin tilassa laitteittain.
2. Tilahistoria näyttää aikajärjestyksessä jokaisen tilanmuutoksen, esimerkiksi milloin peli tuli hyllyyn tai meni läpi.
3. Pelilistassa näkyvät laite, tähtiarvio ja oma kommentti, ja tila vaihtuu yhdellä napilla.
4. Puhelimellakin tila vaihtuu yhdellä napautuksella, ja muutos tallentuu tilahistoriaan.

Projektin 5 vaihetta

Mitä syntyy missäkin vaiheessa?



1. Suunnitelma ja julkaistu runko · työviikot 1–3. Puret toimeksiannon kysymyksiksi, teet ohjaajan kuittaaman suunnitelman ja viet tyhjän palvelun julkiseen osoitteeseen. Julkaisu tehdään heti, koska sen ongelmat on halvinta korjata, kun rikottavaa on vähän.

2. Oma hylly: pelit ja tilanvaihto · työviikot 4–6. Rakennat pelilistan tietokannasta, oman lisäyslomakkeen validointineen ja yhden napin tilanvaihdon, joka kirjaa tilahistorian. Asiakkaan tärkein toive toimii ennen kuin palveluun tulee muita käyttäjiä.

3. Jäsenet, profiilit ja katselmointi · työviikot 7–10. Lisäät rekisteröitymisen, kirjautumisen ja omistajuustarkistuksen, julkisen profiilin sekä haun. Asiakkaan edustaja kokeilee koko polkua työviikolla 10, ja hänen palautteensa ohjaa seuraavaa vaihetta.

4. Palaute, laatu ja käyttöönotto-ohje · työviikot 11–15. Toteutat palautteen tärkeimmän muutoksen ja teet palvelusta puhelimella ja näppäimistöllä toimivan. Testaat kaiken tuotantoa vasten, siistit koodin, arvioit tietoturvan ja kirjoitat käyttöönotto-ohjeen.

5. Julkaisu ja näyttö · työviikot 16–18. Ulkopuolinen testaaja ottaa julkaisuehdokkaan käyttöön pelkän kirjallisen ohjeen avulla. Korjaat estävät virheet, julkaiset v1.0:n ja esittelet työsi näytössä.

Viikon työvaihe on tämän sivuston työohje. Kun työvaihe muuttaa palvelua, kirjaa muutos GitHub-issueksi hyväksymiskriteereineen ja tee se Työtapa-sivun kuudella askeleella. Testin tulos kirjataan testimatriisiin heti, viikon yhteenveto projektipäiväkirjaan viikon lopussa.

Aikataulu

Vko	Ajoitus	Viikon aihe	Vaihe
1	Työviikko 1 / 18	Aloitus	1
2	Työviikko 2 / 18	Suunnitelma ja tietomalli	1
3	Työviikko 3 / 18	Julkaistu runko	1
4	Työviikko 4 / 18	Pelilista tietokannasta	2
5	Työviikko 5 / 18	Pelin lisäys ja validointi	2
6	Työviikko 6 / 18	Tilat ja tilahistoria	2
7	Työviikko 7 / 18	Käyttäjät ja kirjautuminen	3
8	Työviikko 8 / 18	Profiilisivu	3
9	Työviikko 9 / 18	Haku ja suodatus	3
10	Työviikko 10 / 18	Asiakaskatselmointi	3
11	Työviikko 11 / 18	Palautemuutos	4
12	Työviikko 12 / 18	Responsiivisuus ja saavutettavuus	4
13	Työviikko 13 / 18	Testausviikko	4
14	Työviikko 14 / 18	Laatuviikko	4
15	Työviikko 15 / 18	Dokumentaatio	4
16	Työviikko 16 / 18	Julkaisuehdokas ja julkaisutestaus	5
17	Työviikko 17 / 18	Julkaisu v1.0	5
18	Työviikko 18 / 18	Näyttö	5

Palautus viimeistään työviikon 18 lopussa. Sivusto: tehtävien tarkat ohjeet, toteutusavut ja projektipäiväkirja.

Vaihe 1 – Suunnitelma ja julkaistu runko

Työviikko 1 / 18 – Aloitusp

Yhteys kokonaisprojektiin: Projekti alkaa toimeksiannosta ja työkaluista, ennen ensimmäistä komponenttia. Tällä viikolla puret toimeksiannon kysymyksiksi ja rakennat ympäristön, jossa työ tallentuu muuallekin kuin omalle koneelle. Kysymysten vastauksista tulee työviikolla 2 suunnitelma, ja sama repository kerää kaikki työnäytteet näyttöön asti.

Viikon tavoite: Ohjaaja avaa repositoryn toisella koneella ja saa sovellusrungon käyntiin pelkän README:n ohjeilla.

Työvaihe 1 / 4 · Pura toimeksianto kysymyslistaksi

- ☐ Lue toimeksianto. Lue Pelikellari ry:n toimeksianto kokonaan ja alleviivaa vaatimukset, esimerkiksi tilanvaihto yhdellä napilla ja vierailijan näkymä ilman tunnuksia.
- ☐ Merkitse epäselvyydet. Merkitse kohdat, joista et tiedä, mitä asiakas tarkoittaa tai milloin toiminto on valmis.
- ☐ Kirjoita kysymyslista. Kirjoita tiedostoon `project-docs/kysymykset.md` vähintään kahdeksan kysymystä, joihin et tiedä vastausta.
- ☐ Tarkista, että kysymys pakottaa päätöksen. Muotoile kysymykset niin, että vastaus muuttaa jotain toteutuksessa. Katso viikon esimerkki.
- ☐ Vie lista ohjaajalle. Käy lista läpi ohjaajan kanssa. Hän toimii asiakkaan sijaisena, kunnes asiakkaan edustaja on nimetty. Kirjaa vastaukset kysymysten alle työviikon 2 tarinoita varten.

Valmis kun: `project-docs/kysymykset.md` sisältää vähintään kahdeksan kysymystä, ja lista on käyty läpi ohjaajan kanssa.

Tallenna työnäyte: `project-docs/kysymykset.md` repositoryyn. Epäselvät kohdat ja tärkein kysymys työviikon 1 päiväkirjaan.

Työvaihe 2 / 4 · Todenna työkalut ja luo Vite + React -projekti

- ☐ Asenna työkalut. Asenna Node.js:n LTS-versio eli pitkään tuettu vakaa versio, Git ja editori, esimerkiksi VS Code.
- ☐ Ota versiot talteen. Aja `node -v` ja `git --version` ja kopioi tulosteet työviikon 1 päiväkirjaan.
- ☐ Luo kansiot. Luo projektin juureen kansiot `client/`, `server/` ja `project-docs/`. Kaikki dokumentaatio menee `project-docs`-kansioon.
- ☐ Luo React-projekti. Aja `client`-kansiossa `npm create vite@latest` ja valitse React.
- ☐ Käynnistä kehityspalvelin. Aja `npm install` ja `npm run dev` ja avaa sovellusrunko selaimessa.

Valmis kun: Versiotulosteet ovat tallessa, ja `npm run dev` käynnistää sovellusrungon `client`-kansiossa.

Tallenna työnäyte: Versiotulosteet työviikon 1 päiväkirjaan. Projektirunko viedään repositoryyn seuraavassa työvaiheessa.

Työvaihe 3 / 4 · Vie projekti etärepositoryyn

- ☐ Kirjoita README. Kirjoita `README.md`: mikä PeliHylly on ja miten sovellusrunko käynnistetään.
- ☐ Lisää `.gitignore`. Lisää `.gitignore`, joka estää `node_modules`-kansion ja `.env`-tiedostot pääsemästä repositoryyn.
- ☐ Tee ensimmäinen commit. Tee ensimmäinen commit kuvaavalla viestillä, esimerkiksi "Vite + React -runko ja README".
- ☐ Vie etärepositoryyn. Luo etärepository GitHubiin ja vie commitit sinne komennolla `git push`.
- ☐ Kokeile kloonausta. Kloonaa repository toiseen kansioon ja tarkista, että runko käynnistyy pelkän README:n ohjeilla.

Valmis kun: Repository on GitHubissa, README kertoo, mikä projekti on, ja kloonattu kopio käynnistyy README:n ohjeilla.

Tallenna työnäyte: Repositoryn osoite ja ensimmäisen commitin tunniste työviikon 1 päiväkirjaan.

Työvaihe 4 / 4 · Käy julkisen repositoryn tarkistuslista ohjaajan kanssa

- ☐ Tarkista historia. Käy repositoryn tiedostot ja commit-historia läpi. Niissä ei saa olla henkilötietoja, salasanoja eikä avaimia.
- ☐ Sovi tekijänimestä. Sovi ohjaajan kanssa, millä nimellä olet tekijänä repositoryssa, ja kirjaa se suunnitelman Tekijä-kenttään.
- ☐ Tarkista suostumustarve. Kysy ohjaajalta, tarvitaanko huoltajan suostumus. Alaikäisellä se hoidetaan ohjaajan kautta.
- ☐ Merkitse avoimet asiat. Repositoryn julkisuus ja lisenssi ovat ohjaajan päätöksiä. Jätä ne suunnitelmassa avoimiksi, kunnes ne on sovittu.

Valmis kun: Julkisen repositoryn tarkistuslista on käyty ohjaajan kanssa, ja sopimatta olevat asiat on merkitty avoimiksi.

Tallenna työnäyte: Sovitut asiat suunnitelmaan (Tekijä ja ohjaajan kentät). Tarkistuksen tulos työviikon 1 päiväkirjaan.

Viikon lopputarkistus: Ohjaaja avaa repositoryn osoitteen toisella koneella: README kertoo mikä projekti on, ja `npm install && npm run dev` käynnistää sovellusrungon. Kysymyslista on [project-docs/kysymykset.md](#)-tiedostossa.

Työnäyte Git-repositoryyn ennen rastia: Repositoryn osoite, ensimmäisen commitin tunniste, työkalutodennusten tulosteet ja [project-docs/kysymykset.md](#).

Työviikko 2 / 18 – Suunnitelma ja tietomalli

Yhteys kokonaisprojektiin: Työviikon 1 kysymyslista sai vastauksia, ja nyt ne muuttuvat käyttäjätarinoiksi, tietomalliksi ja rajaukseksi. Päätät, mitä tehdään ensin ja miten tieto tallennetaan, koska tilahistoria ja kokoelmat on vaikea lisätä jälkikäteen. Samaa suunnitelmaa vasten rakennat ja tarkistat työtä työviikosta 3 alkaen.

Viikon tavoite: Ohjaaja on kuitannut pakollisen ytimen rajauksen, ja jokaisella sen issuella on hyväksymiskriteerit ja työmääräarvio.

Työvaihe 1 / 4 · Kirjoita käyttäjätarinat ja priorisoi ne ohjaajan kanssa

- ☐ Kirjoita tarinat. Kirjoita tiedostoon `project-docs/kayttajatarinat.md` jokaisesta toiminnosta tarina muodossa ”Jäsenenä haluan..., jotta...”. Käytä työviikon 1 vastauksia.
- ☐ Lisää hyväksymiskriteerit. Kirjoita jokaiseen tarinaan 1–3 havaittavaa kriteeriä, joista ulkopuolinen voi todeta, onko tarina valmis.
- ☐ Luokittele tarinat. Merkitse tarinat: pakollinen ydin (P0), tärkeä jatko (P1) tai valinnainen lisä (P2). Profiiliviestit kuuluvat P2:een.
- ☐ Priorisoi ohjaajan kanssa. Käy luokittelu läpi ohjaajan kanssa ja muuta sitä, jos rajauksesta sovitaan toisin.
- ☐ Pyydä kirjallinen kuittaus. Avaa GitHubiin issue ”P0-rajaus”, linkitä siihen tarinatiedosto ja pyydä ohjaajalta kuittaus issue-kommenttina.

Valmis kun: Jokaisella tarinalla on hyväksymiskriteerit ja prioriteetti, ja ohjaaja on kuitannut pakollisen ytimen (P0) rajauksen issue-kommentissa.

Tallenna työnäyte: `project-docs/kayttajatarinat.md` repositoryyn. Linkki ohjaajan kuittaukseen ja P0:n ulkopuolelle jääneet tarinat työviikon 2 päiväkirjaan.

Työvaihe 2 / 4 · Piirrä tietomalli, sivukartta ja rautalangat

- ☐ Piirrä tietomalli. Piirrä taulut, avaimet, viiteavaimet ja tilahistorian rakenne. Viikon toteutusavun kentät ovat esimerkki: päättää omat.
- ☐ Nimeä tilat alustavasti. Kirjaa pelin tilat, esimerkiksi hyllyssä, kesken ja läpi. Nimet testataan käytännössä työviikolla 6.
- ☐ Piirrä sivukartta. Piirrä näkymät ja siirtymät niiden välillä, esimerkiksi etusivu, pelit, pelin sivu, profiili ja kirjautuminen.
- ☐ Piirrä rautalangat. Piirrä rautalangat eli karkeat luonnokset näkymien rakenteesta paperille tai työkalulla. Tee profiilisivusta tarkin.
- ☐ Kirjaa suunnitelmaan. Kirjoita tietomallin ja sivukartan tiivistelmä suunnitelman kenttiin ja tallenna kuvat `project-docs/-kansioon`.

Valmis kun: Tietomallissa näkyvät taulut ja avaimet, ja profiilin rautalanka on niin tarkka, että toteutusta voi verrata siihen työviikolla 8.

Tallenna työnäyte: Tietomallikaavio, sivukartta ja rautalangat kuvina `project-docs/-kansioon`. Tiivistelmät suunnitelman kenttiin Tietomalli ja Sivukartta.

Työvaihe 3 / 4 · Vertaile tietovarastot ja perustele SQLite

- ☐ Valitse kriteerit. Päättää omat vertailukriteerit, esimerkiksi datan rakenne, monta-moneen-suhteet, samanaikainen kirjoitus ja julkaisun helppous.
- ☐ Vertaile SQLiteä muihin. Täytä vertailutaulukko: SQLite vähintään yhtä vaihtoehtoa vasten, esimerkiksi JSON-tiedosto tai palvelintietokanta. JSON-tiedosto on tavallinen tekstitiedosto, johon tieto tallennetaan.
- ☐ Perustele SQLite. Kirjoita 2–3 virkkeen perustelu, miksi SQLite sopii PeliHyllyyn. Viittaa kriteereihisi.
- ☐ Kirjaa suunnitelmaan. Kirjoita vertailu ja perustelu suunnitelman kenttään Tietovarasto ja perustelu.
- ☐ Kerro tulos ohjaajalle. Kerro vertailun tulos ohjaajalle. Jos vertailu puoltaa selvästi muuta, sovi poikkeamasta ohjaajan kanssa ennen työviikkoa 3.

Valmis kun: Suunnitelmassa on vertailu, jossa SQLiteä verrataan omilla kriteereillä vähintään yhteen vaihtoehtoon, ja perustelu sovitulle valinnalle.

Tallenna työnäyte: Vertailu ja perustelu suunnitelman kenttään Tietovarasto ja perustelu. Perustelu omin sanoin työviikon 2 päiväkirjaan.

Työvaihe 4 / 4 · Pilko pakollinen ydin issueiksi ja arvioi työmäärät

- ☐ Pilko tarinat. Pilko jokainen pakollisen ytimen (P0) tarina GitHub-issueiksi. Yksi issue on yksi rajattu muutos, yleensä puolen tai yhden päivän työ.
- ☐ Kirjaa kriteerit issueen. Kopioi issueen ne tarinan hyväksymiskriteerit, joihin issue vastaa.
- ☐ Arvioi työmäärä. Kirjaa jokaiseen issueen työmääräarvio tunteina tai päivinä.
- ☐ Järjestä issue-taulu. Järjestä issueet issue-tilaan siihen järjestykseen, jossa aiot tehdä ne.
- ☐ Sovi aloitus ohjaajan kanssa. Sovi ohjaajan kanssa, mitkä issueet tehdään ensin. Tehtävistä sovitaan näin joka viikko tästä eteenpäin.

Valmis kun: Jokaisella pakollisen ytimen (P0) issueella on hyväksymiskriteerit ja työmääräarvio.

Tallenna työnäyte: Issue-tilan linkki työviikon 2 päiväkirjaan.

Viikon lopputarkistus: Suunnitelma on `project-docs/suunnitelma.md`-tiedostossa, ohjaaja on kuitannut P0-rajauksen kirjallisesti (issue-kommentti), ja jokaisella P0-issueella on työmääräarvio.

Työnäyte Git-repositoryyn ennen rastia: `project-docs/suunnitelma.md`, tietomallikaavio, rautalangat, issue-tila arvioineen ja ohjaajan kirjallinen kuittaus P0-rajauksesta.

Työviikko 3 / 18 – Julkaistu runko

Yhteys kokonaisprojektiin: Työviikon 2 sivukartta ja tietomalli muuttuvat nyt koodiksi: reitit vastaavat sivukarttaa, ja migraatioskripti luo tietomallin taulut. Koko putki viedään heti julkiseen osoitteeseen, koska julkaisun ongelmat, kuten ympäristömuutujat ja SQLiten levypolku, on halvinta korjata, kun rikottavaa on vähän. Samaan osoitteeseen tulevat työviikosta 4 alkaen pelit ja työviikolla 17 versio v1.0.

Viikon tavoite: Kuka tahansa voi avata PeliHyllyn rungon julkisesta osoitteesta, ja `/api/health` vastaa tuotannossa.

Työvaihe 1 / 4 · Rakenna reitit react-routerilla ja kirjoita k2-muistio

- ☐ Kirjoita k2-muistio. Kirjoita `project-docs/k2-muistio.md`: mitä React ratkaisee (komponentit, tila, renderointi), mitä ei (reititys, palvelin, tietovarasto) ja mitä siitä seuraa PeliHyllylle.
- ☐ Asenna react-router. Asenna react-router client-kansioon ja kirjaa riippuvuus perusteluineen k2-muistioon.
- ☐ Rakenna kolme reittiä. Tee sivut Etusivu, Pelit ja Profiili kansioon `client/src/sivut/` ja reitit tiedostoon `App.jsx`. Sisällöksi riittää paikanpitäjä.
- ☐ Lisää navigaatio. Lisää navigaatio, jolla pääsee jokaiselle kolmelle sivulle.
- ☐ Vertaa sivukarttaan. Tarkista, että reitit vastaavat työviikon 2 sivukarttaa. Kirjaa poikkeama suunnitelman kenttään Sivukartta ja komponenttijako.

Valmis kun: Kolme reittiä toimii navigaatiosta, ja k2-muistio kertoo omin sanoin, miksi reititys tarvitsee erillisen kirjaston.

Tallenna työnäyte: `project-docs/k2-muistio.md` ja reitit commitilla repositoryyn.

Työvaihe 2 / 4 · Pystytä Express-palvelin ja tietokannan skeema

- ☐ Luo Express-projekti. Luo server-kansioon Express-projekti ja tiedosto `server/index.js`, joka käynnistää palvelimen.
- ☐ Tee terveystarkistus. Toteuta reitti `/api/health`, joka palauttaa version ja tietokantayhteyden tilan. Kokeile sitä selaimessa.
- ☐ Kytke SQLite. Tee tiedosto `server/db.js`, joka avaa SQLite-tietokannan. Lue tietokannan polku ympäristömuuttujasta.
- ☐ Kirjoita migraatioskripti. Kirjoita `server/migrate.js`, joka luo työviikon 2 tietomallin taulut eli tietokannan skeeman ja tulostaa, montako taulua syntyi.
- ☐ Aja migraatio. Aja migraatio tyhjään tietokantaan ja tarkista tulosteesta, että taulujen määrä vastaa tietomallia.

Valmis kun: `/api/health` vastaa paikallisesti, ja migraatio luo tietomallin taulut tyhjään tietokantaan.

Tallenna työnäyte: `server/`-kansion tiedostot commitilla repositoryyn. Migraation tuloste työviikon 3 päiväkirjaan.

Työvaihe 3 / 4 · Vertaile julkaisualustat ja julkaise runko

- ☐ Vertaile alustat. Täytä vertailutaulukko: Render, Railway ja Fly.io. Kriteereinä ilmainen taso, levyn pysyvyys SQLite-tiedostolle, ympäristömuuttujat ja käyttöönoton työmäärä.
- ☐ Tee päätös. Valitse alusta ja kirjaa perustelu suunnitelman kenttään Julkaisualusta. Tilin omistajuus sovitaan ohjaajan kanssa.
- ☐ Kokoa tuotantobuild. Aja `client`-kansiossa `npm run build` ja tarkista, että Express tarjoilee `client/dist`-kansion.
- ☐ Aseta ympäristö. Aseta salaisuudet ja tietokannan polku alustan ympäristömuuttujiksi. Tarkista, että SQLite-tiedosto on pysyvällä levyllä.
- ☐ Julkaise ja tarkista. Julkaise ja avaa julkinen osoite sekä `/api/health` puhelimella mobiiliverkossa.
- ☐ Kirjaa osoite. Lisää julkinen osoite README:hen ja ota julkaisulokista kuvakaappaus.

Valmis kun: Julkinen osoite näyttää navigoitavan rungon, ja `/api/health` vastaa tuotannossa.

Tallenna työnäyte: Julkaisulokin kuvakaappaus `project-docs/-`-kansioon, osoite README:hen ja alustojen vertailu työviikon 3 päiväkirjaan.

Työvaihe 4 / 4 · Lähetä asiakkaalle tilannekatsaus

- ☐ Valitse vastaanottaja. Lähetä katsaus asiakkaan edustajalle. Jos häntä ei ole vielä nimetty, ohjaaja toimii asiakkaan sijaisena ja vastaanottaa katsauksen.
- ☐ Kirjoita kolme virkettä. Kerro ilman teknistä jargonia, mitä nyt on valmiina, mitä seuraavaksi tehdään ja mistä osoitteesta palvelun näkee.
- ☐ Poista jargon. Lue viesti asiakkaan silmin ja vaihda sanat kuten rajapinta, build tai migraatio arkikielelle.
- ☐ Lähetä ja tallenna. Lähetä viesti ja tallenna se tiedostoon `project-docs/viestit.md`. Kirjaa vastaanottajasta vain rooli, ei nimeä.

Valmis kun: Tilannekatsaus on lähetetty ja tallennettu `project-docs/viestit.md`-tiedostoon.

Tallenna työnäyte: `project-docs/viestit.md` commitilla repositoryyn.

Viikon lopputarkistus: Julkinen osoite näyttää navigoitavan rungon, `/api/health` vastaa tuotannossa, ja tilannekatsausviesti on lähetetty ja tallennettu `project-docs/viestit.md`-tiedostoon.

Työnäyte Git-repositoryyn ennen rastia: Julkinen osoite, julkaisuloki tai kuvakaappaus, k2-muistio ja tilannekatsausviesti `project-docs/viestit.md`-tiedostossa.

Vaihe 2 – Oma hylly: pelit ja tilanvaihto

Työviikko 4 / 18 – Pelilista tietokannasta

Yhteys kokonaisprojektiin: Työviikon 3 tyhjät reitit saavat nyt sisällön. Sama tietokanta, jonka migraatio loi, tarjoillaan oman rajapintareitin kautta React-komponenteille, ja virhetilat käsitellään heti. Lukuyhteys on pohja, jonka päälle työviikolla 5 tulee pelin lisäys.

Viikon tavoite: Käyttäjä näkee pelilistan tietokannasta, ja jos palvelin on alhaalla, hän saa selkeän virheilmoituksen tyhjän sivun sijaan.

Työvaihe 1 / 4 · Tee siemendata ja pelilistan rajapintareitti

- ☐ Kirjoita siemendata. Kirjoita `server/seed.js`, joka lisää tietokantaan omat esimerkkipelisi. Kirjoita pelit itse, älä kopioi ulkopuolista tietokantaa.
- ☐ Aja siemendata. Aja skripti ja tarkista `sqlite3`-kyselyllä, että pelit ovat tietokannassa.
- ☐ Tee lukureitti. Toteuta `GET /api/games`, joka lukee pelit tietokannasta ja palauttaa ne JSON-muodossa.
- ☐ Kokeile reittiä. Avaa reitti selaimessa tai curlilla ja vertaa vastausta siemendataan.

Valmis kun: `GET /api/games` palauttaa siemendatan pelit tietokannasta.

Tallenna työnäyte: Siemendataskripti ja reitti commitilla repositoryyn.

Työvaihe 2 / 4 · Rakenna pelilista komponenteista omalla fetch-hookilla

- ☐ Tee oma hookki. Kirjoita hookki `usePelit()`, joka hakee pelit `useEffectin` ja `fetchin` avulla ja palauttaa datan, lataustilan ja virheen.
- ☐ Rakenna lista ja kortti. Tee komponentit `PeliLista` ja `PeliKortti`. Lista saa pelit propsina ja piirtää kortin jokaiselle pelille.
- ☐ Erota tilamerkki. Tee `TilaMerkki`-komponentti, joka näyttää pelin tilan. Kortti ei itse päästä tilan ulkoasua.
- ☐ Kytke Pelit-sivuun. Käytä hookkia `PelitSivu`-komponentissa ja näytä lista reitillä, jonka teit työviikolla 3.
- ☐ Julkaise ja tarkista. Julkaise muutos ja tarkista, että lista näkyy tuotannossa tietokannan datasta.

Valmis kun: Pelilista näkyy tuotannossa tietokannan datasta, ja jokaisella komponentilla on yksi vastuu.

Tallenna työnäyte: Komponentit ja hookki commitilla. Vastuunjako omin sanoin työviikon 4 päiväkirjaan.

Työvaihe 3 / 4 · Käsittele lataus-, virhe- ja tyhjä tila

- ☐ Näytä lataustila. Näytä käyttäjälle viesti tai merkki silloin, kun pelejä haetaan.
- ☐ Tarkista vastaus. Tarkista hookissa `response.ok` ja palauta virhe, jos palvelin vastaa virheellä.
- ☐ Kirjoita virheviesti. Näytä virhe käyttäjän kielellä, esimerkiksi "Pelin haku ei onnistunut. Yritä hetken päästä uudelleen."
- ☐ Käsittele tyhjä lista. Näytä oma viesti, jos pelejä ei ole. Älä jätä sivua tyhjäksi.
- ☐ Sammuta palvelin. Sammuta palvelin, lataa sivu uudelleen ja ota kuvakaappaus sekä normaalitilasta että virhetilasta.

Valmis kun: Kun palvelin on alhaalla, sivu näyttää virheilmoituksen eikä jää tyhjäksi. Molemmista tiloista on kuvakaappaus.

Tallenna työnäyte: Kuvakaappaukset normaali- ja virhetilasta `project-docs/-kansioon` ja muutokset commitilla.

Työvaihe 4 / 4 · Kirjaa ja aja testitapaukset T01 ja T02

- ☐ Luo testimatriisi. Luo tiedosto `project-docs/testimatriisi.md`, jossa on sarakkeet tunnus, luokka, askeleet, odotettu tulos, toteutunut tulos ja viikko.
- ☐ Kirjaa testitapaus T01. Normaali käyttö: pelilista latautuu ja näyttää siemendatan. Kirjaa odotettu tulos ennen ajoa.
- ☐ Kirjaa testitapaus T02. Virhetilanne: palvelin on alhaalla, ja käyttäjä näkee virheilmoituksen. Kirjaa odotettu tulos ennen ajoa.
- ☐ Aja ja kirjaa tulokset. Aja molemmat testitapaukset ja kirjaa toteutuneet tulokset matriisiin.
- ☐ Tee commit. Tee commit, jonka viesti kertoo, mitkä testitapaukset lisättiin.

Valmis kun: Testimatriisissa ovat testitapaukset T01 ja T02: odotettu tulos kirjattuna ennen ajoa ja toteutunut tulos ajon jälkeen.

Tallenna työnäyte: `project-docs/testimatriisi.md` commitilla. Odotetut ja toteutuneet tulokset lyhyesti työviikon 4 päiväkirjaan.

Viikon lopputarkistus: Lista renderöityy tuotannossa tietokannan datasta; kun API on alhaalla, sivu näyttää virheilmoituksen eikä jää tyhjäksi; molemmista kuvakaappaus.

Työnäyte Git-repositoryyn ennen rastia: Commitit, kuvakaappaus normaali- ja virhetilasta sekä testikirjaukset T01–T02.

Työviikko 5 / 18 – Pelin lisäys ja validointi

Yhteys kokonaisprojektiin: Työviikolla 4 pelejä vain luettiin, nyt niitä myös kirjoitetaan. Rakennat lomakkeen itse ja tarkistat syötteet sekä selaimessa että palvelimella, koska rajapintaa voi kutsua suoraan selaimen ohi. Tähtiariot ja kommentit syntyvät lomakkeen kenttänä, ja samaa lomaketta käytetään myöhemmin pelin muokkaukseen.

Viikon tavoite: Jäsen lisää pelin tähtiarioineen ja kommentteineen, ja virheellinen syöte hylätään kentittäisellä viestillä myös suorassa kutsussa.

Työvaihe 1 / 4 · Rakenna pelin lisäyslomake itse

- ☐ Suunnittele kentät. Pääta kentät nimi, laite, tila, tähdet ja kommentti sekä niiden pakollisuus suunnitelman mukaan.
- ☐ Kirjaa säännöt ensin. Täytä validointisääntötaulukko ennen koodia: kenttä, sääntö, virheviesti ja missä sääntö tarkistetaan.
- ☐ Rakenna lomake. Tee PeliLomake-komponentti ilman valmista käyttöliittymäkirjastoa. Kenttien arvot ovat Reactin tilassa, eli lomake on kontrolloitu.
- ☐ Kytke labelit. Kytke jokainen label omaan kenttäänsä. Tähtiartion pitää toimia myös näppäimistöllä.
- ☐ Lisää selainvalidointi. Toteuta taulukon säännöt selaimessa ennen kuin lomake lähettää pyynnön.

Valmis kun: Lomake toimii näppäimistöllä, jokainen kenttä on kytketty labeliinsa, ja validointisäännöt on kirjattu taulukkoon ennen koodia.

Tallenna työnäyte: Validointisääntötaulukko `project-docs/-`-kansioon ja lomake commitilla repositoryyn.

Työvaihe 2 / 4 · Toteuta pelin lisäys ja palvelininvalidointi

- ☐ Tee lisäysreitti. Toteuta `POST /api/games`, joka tallentaa pelin tietokantaan ja palauttaa lisätyn pelin.
- ☐ Toteuta samat säännöt. Tarkista Expressissä samat säännöt kuin selaimessa sääntötaulukon mukaan.
- ☐ Palauta virhelista. Palauta virheelliseen pyyntöön virhekoodi 400 (virheellinen pyyntö) ja virhelista kentittäin, esimerkiksi `{ nimi: "..." }`.
- ☐ Todista curlilla. Lähetä curlilla selaimen ohi pyyntö ilman pakollisia kenttiä ja tallenna tuloste. Kirjaa odotus ennen ajoa.

Valmis kun: Tyhjä pyyntö curlilla saa vastaukseksi virhekoodin 400 ja virhelistan, ja onnistunut lisäys tallentuu tietokantaan.

Tallenna työnäyte: curl-tulosteet `project-docs/-`-kansioon ja reitti commitilla repositoryyn.

Työvaihe 3 / 4 · Näytä virheviestit kenttien vieressä

- ☐ Käytä samaa virhemuotoa. Käytä samaa virhemuotoa selaimessa ja palvelimella, jotta palvelimen virhelista näkyy samoissa kohdissa.
- ☐ Näytä viesti kentän vieressä. Näytä kunkin kentän virheviesti heti kentän alla, ei vain sivun yläreunassa.
- ☐ Kytke viesti kenttään. Kytke virheviesti kenttään `aria-describedby-attribuutilla`.
- ☐ Kokeile palvelimen virhettä. Ohita selainvalidointi hetkeksi ja tarkista, että palvelimen palauttama virhe näkyy oikean kentän vieressä.

Valmis kun: Virheviesti näkyy oikean kentän vieressä sekä selaimen että palvelimen hylkäämässä pyynnössä, ja viesti on kytketty kenttään.

Tallenna työnäyte: Muutokset commitilla ja kuvakaappaus kentittäisistä virheviesteistä `project-docs/-`-kansioon.

Työvaihe 4 / 4 · Kirjaa ja aja testitapaukset T03–T05

- ☐ Kirjaa testitapaus T03. Normaali käyttö: onnistunut lisäys. Kirjaa testimatriisiin odotettu tulos ennen ajoa.
- ☐ Kirjaa testitapaus T04. Raja: ylipitkä nimi, esimerkiksi 101 merkkiä. Kirjaa odotettu tulos ennen ajoa.
- ☐ Kirjaa testitapaus T05. Virhetilanne: suora kutsu curlilla ilman pakollisia kenttiä. Kirjaa odotettu tulos ennen ajoa.
- ☐ Aja ja vertaa. Aja tapaukset selaimessa ja curlilla ja kirjaa toteutuneet tulokset. Molempien pitää hylätä virheellinen pyyntö samalla säännöllä.

Valmis kun: Testitapaukset T03–T05 on kirjattu odotuksineen ennen ajoa, ja toteutuneet tulokset on kirjattu matriisiin.

Tallenna työnäyte: `project-docs/testimatriisi.md` commitilla. Validointisäännöt ja curl-ajon havainnot työviikon 5 päiväkirjaan.

Viikon lopputarkistus: Tyhjä nimi, ylipitkä nimi ja onnistunut lisäys käyttäytyvät ennalta kirjatun odotuksen mukaan sekä selaimessa että suoralla API-kutsulla (curl-tuloste talteen).

Työnäyte Git-repositoryyn ennen rastia: Commitit, validointisääntötaulukko, curl-tulosteet ja testikirjaukset T03–T05.

Työviikko 6 / 18 – Tilat ja tilahistoria

Yhteys kokonaisprojektiin: Työviikolla 5 peli saatiin hyllyyn, nyt se saa elämän. Työviikon 2 tietomallin tilahistoriataulu otetaan käyttöön, ja tilojen nimet testataan käytännössä. Kun tilanvaihto toimii, profiili voi työviikolla 8 koota historiasta jäsenen tilastot.

Viikon tavoite: Jäsen vaihtaa pelin tilan yhdellä napilla, ja jokainen muutos näkyy pelin tilahistoriassa.

Työvaihe 1 / 4 · Määrittele pelin tilat ja sallitut siirtymät

- ☐ Nimeä tilat. Nimeä tilat itse, esimerkiksi hyllyssä, kesken, läpi ja kaikki tehty. Älä kopioi aihekuvauksesta miettimättä.
- ☐ Täytä siirtymätaulukko. Kirjaa taulukkoon, mistä tilasta mihin pääsee ja mihin ei.
- ☐ Perustele estot. Kirjoita jokaiselle estetylle siirtymälle yhden virkkeen perustelu.
- ☐ Päivitä suunnitelma. Päivitä suunnitelman kenttä Tietomalli ja tilojen nimet ja kirjaa, mikä muuttui työviikon 2 tietomallista.

Valmis kun: Suunnitelmassa ovat tilojen nimet ja siirtymätaulukko, jossa jokainen estetty siirtymä on perusteltu.

Tallenna työnäyte: Siirtymätaulukko `project-docs/-kansioon` ja päivitetty `project-docs/suunnitelma.md` commitilla.

Työvaihe 2 / 4 · Toteuta tilanvaihto, joka kirjaa historiarivin samassa transaktiossa

- ☐ Tee tilanvaihtoreitti. Toteuta rajapintareitti, joka vaihtaa yhden pelin tilan.
- ☐ Tarkista siirtymä. Hylkää siirtymä, jota siirtymätaulukko ei salli, ja palauta selkeä virhe.
- ☐ Kirjoita transaktio. Päivitä `games-tilun` tila ja lisää `status_history`-tauluun vanha tila, uusi tila ja aikaleima samassa transaktiossa.
- ☐ Kokeile virhettä. Aiheuta tarkoituksella virhe historiarivin lisäyksessä ja tarkista, ettei myöskään tila muuttunut.

Valmis kun: Tilanvaihto ja historiarivi tallentuvat yhdessä, ja epäonnistunut vaihto ei jätä voimaan tilaa eikä riviä.

Tallenna työnäyte: Reitti commitilla repositoryyn. Miten varmistit transaktion, työviikon 6 päiväkirjaan.

Työvaihe 3 / 4 · Lisää tilanappi, tilahistoria ja pelaan parhaillaan -merkintä

- ☐ Tee pelin sivu. Tee pelin sivu, jolla näkyvät pelin tiedot ja nykyinen tila.
- ☐ Lisää tilanappi. Lisää pelin sivulle nappi, joka vaihtaa tilan yhdellä painalluksella ja päivittää näkymän heti.
- ☐ Näytä historia. Näytä pelin tilahistoria aikajärjestyksessä, uusin ylimpänä.
- ☐ Lisää pelaan parhaillaan -merkintä. Toteuta merkintä, jonka jäsen voi laittaa päälle ja pois.
- ☐ Tarkista julkaistussa versiossa. Vaihda tila julkaistussa versiossa ja tarkista, että uusi tila ja historiarivi näkyvät heti.

Valmis kun: Tilan vaihto yhdellä napilla näkyy heti pelin sivulla ja historiassa, ja pelaan parhaillaan -merkintä tallentuu.

Tallenna työnäyte: Muutokset commitilla ja kuvakaappaus pelin sivusta historioineen `project-docs/-kansioon`.

Työvaihe 4 / 4 · Kirjaa ja aja testitapaus T06 tietokantaa vasten

- ☐ Kirjaa testitapaus T06. Normaali käyttö: tilanvaihto luo historiarivin, jonka vanha ja uusi tila täsmäävät. Kirjaa odotettu tulos testimatriisiin ennen ajoa.
- ☐ Aja kysely ennen. Aja todennuskysely `sqlite3`-komennolla ennen tilanvaihtoa ja tallenna tuloste.
- ☐ Vaihda tila kerran. Vaihda yhden pelin tila käyttöliittymästä.
- ☐ Aja kysely jälkeen. Aja sama kysely ja tarkista, että syntyi täsmälleen yksi rivi oikealla aikaleimalla.
- ☐ Kirjaa tulos. Kirjaa toteutunut tulos testimatriisiin ja liitä molemmat tulosteet päiväkirjaan.

Valmis kun: Testitapaus T06 on kirjattu odotuksineen ennen ajoa, ja kyselyn tulosteet ennen ja jälkeen todistavat yhden täsmäävän historiarivin.

Tallenna työnäyte: `project-docs/testimatriisi.md` commitilla. `sqlite3`-tulosteet ennen ja jälkeen työviikon 6 päiväkirjaan.

Viikon lopputarkistus: Tilan vaihto näkyy heti pelin sivulla, ja `status_history`-taulussa on rivi, jonka aikaleima ja vanha→uusi tila täsmäävät; suora tietokantakysely liitetty päiväkirjaan.

Työnäyte Git-repositoryyn ennen rastia: Commitit, siirtymätaulukko, `sqlite3`-kyselyn tuloste ennen ja jälkeen sekä T06.

Vaihe 3 – Jäsenet, profiilit ja katselmointi

Työviikko 7 / 18 – Käyttäjät ja kirjautuminen

Yhteys kokonaisprojektiin: Työviikoilla 4–6 kuka tahansa pystyi muokkaamaan mitä tahansa. Nyt pelit saavat omistajan, ja aiemmat muokkausreitit suojataan omistajuustarkistuksella. Kirjautuminen on edellytys julkiselle profiilille, jonka rakennat työviikolla 8.

Viikon tavoite: Jäsen rekisteröityy ja kirjautuu, ja vain pelin omistaja voi muokata peliä, myös suoralla rajapintakutsulla.

Työvaihe 1 / 4 · Vertaile istunto ja token ohjaajan kanssa

- ☐ Valitse kriteerit. Kirjaa omat kriteerit, esimerkiksi palvelinten määrä, tietoturva, toteutuksen työmäärä ja uloskirjautuminen.
- ☐ Vertaile kaksi tapaa. Kirjoita vertailumuistio: istuntopohjainen kirjautuminen ja token omia kriteereitäsä vasten.
- ☐ Päätä ohjaajan kanssa. Käy vertailu läpi ohjaajan kanssa ja tehkää päätös yhdessä. Pyydä ohjaajalta kommentti muistioon.
- ☐ Kirjaa päätös. Kirjoita päätös perusteluineen suunnitelman kenttään Autentikointitapa.

Valmis kun: Vertailumuistiossa ovat omat kriteerit, päätös ja ohjaajan kommentti.

Tallenna työnäyte: Vertailumuistio `project-docs/-kansioon` ja päätös `project-docs/suunnitelma.md`-tiedostoon commitilla.

Työvaihe 2 / 4 · Toteuta rekisteröityminen salasanan hashilla

- ☐ Tee käyttäjätaulu. Tee migraatio `users`-taululle: käyttäjänimi, salasanan hash, sähköposti ja valinnainen profiilikuva.
- ☐ Hashaa salasana. Tallenna salasana `bcrypt`-hashina eli yksisuuntaisesti salattuna. Selväkielinen salasana ei tallennu koskaan.
- ☐ Tee rekisteröitymisreitti. Toteuta `POST /api/users` validointineen samaan tapaan kuin työviikon 5 lomakkeessa.
- ☐ Tarkista kanta ja lokit. Rekisteröi testikäyttäjä ja tarkista tietokannasta ja palvelinlokista, ettei selväkielistä salasanaa näy missään.

Valmis kun: Rekisteröityminen toimii, ja tietokannassa ja lokeissa näkyy vain salasanan hash.

Tallenna työnäyte: Muutokset commitilla ja tietokantatarkistuksen tuloste työviikon 7 päiväkirjaan.

Työvaihe 3 / 4 · Toteuta kirjautuminen ja suojaa muokkausreitit

- ☐ Toteuta kirjautuminen. Toteuta `POST /api/login`, joka vertaa salasanaa hashiin. Virheviesti ei paljasta, oliko väärin tunnus vai salasana.
- ☐ Päätä elinkaari. Päätä istunnon tai tokenin voimassaoloaika ja toteuta uloskirjautuminen, joka mitätöi sen.
- ☐ Kirjoita middleware. Kirjoita middleware eli välikäsitteijä, joka tarkistaa jokaisesta muokkauspyynnöstä kirjautumisen (401) ja omistajuuden (403).
- ☐ Suojaa vanhat reitit. Lisää middleware pelin lisäykseen, muokkaukseen ja tilanvaihtoon.
- ☐ Erota näkymät. Tee selaimeen kirjautuneen ja vierailijan näkymät erikseen Reactin contextin avulla.

Valmis kun: Vain kirjautunut omistaja voi muokata peliään. Kirjautumaton pyyntö saa virhekoodin 401 ja toisen käyttäjän pyyntö virhekoodin 403.

Tallenna työnäyte: Muutokset commitilla. Suojattujen reittien lista työviikon 7 päiväkirjaan.

Työvaihe 4 / 4 · Kirjaa ja aja testitapaukset T07 ja T08

- ☐ Kirjaa testitapaus T07. Virhetilanne: väärä salasana. Kirjautuminen estyy, eikä viesti paljasta, kumpi kenttä oli väärin. Kirjaa odotus ennen ajoa.
- ☐ Kirjaa testitapaus T08. Virhetilanne: toisen käyttäjän pelin muokkaus rajapinnan kautta saa virhekoodin 403. Kirjaa odotus ennen ajoa.
- ☐ Hyökkää itseäsi vastaan. Kirjaudu käyttäjänä A, ota istunto tai token talteen ja yritä muokata käyttäjän B peliä curlilla.
- ☐ Kirjaa tulokset. Kirjaa toteutuneet tulokset testimatriisiin ja tallenna curl-tulosteet.

Valmis kun: Testitapaukset T07 ja T08 on ajettu, ja curl-tuloste näyttää virhekoodin 403 toisen käyttäjän peliin.

Tallenna työnäyte: `project-docs/testimatriisi.md` ja curl-tulosteet commitilla. Tulokset työviikon 7 päiväkirjaan.

Viikon lopputarkistus: Toisen käyttäjän pelin muokkausyritys palauttaa 403 (curl-tuloste talteen); tietokannassa ja lokeissa ei näy selväkielistä salasanaa; vertailumuistiossa on ohjaajan kommentti.

Työnäyte Git-repositoryyn ennen rastia: Vertailumuistio ohjaajan kommentilla, commitit, curl-tulosteet sekä T07–T08.

Työviikko 8 / 18 – Profiilisivu

Yhteys kokonaisprojektiin: Työviikkojen 6 ja 7 tilahistoria ja käyttäjät saavat nyt näyteikkunan. Profiili kokoaa saman datan koosteiksi, jotka vierailijakin näkee ilman tunnuksia, ja toteutusta verrataan työviikon 2 rautalankaan. Kun jäsenillä on profiilit, työviikon 9 haku auttaa löytämään pelejä kasvavasta hyllystä.

Viikon tavoite: Vierailija avaa jäsenen profiilin ilman tunnuksia ja näkee, mitä jäsen pelaa nyt, tilastot tiloittain ja laitteittain sekä tilahistorian.

Työvaihe 1 / 4 · Toteuta profiilin kooste SQL-kyselyillä

- ☐ Kirjoita koostekyselyt. Laske pelien määrä tiloittain ja laitteittain SQL:n GROUP BY -kyselyillä.
- ☐ Hae pelaa nyt ja historia. Hae pelit, joilla on pelaan parhaillaan -merkintä, ja jäsenen tilahistoria aikajärjestyksessä.
- ☐ Kokoa yhteen reittiin. Toteuta reitti, joka palauttaa koko profiilin datan yhdellä kutsulla, esimerkiksi `/api/users/:nimi/profiili`.
- ☐ Tarkista luvut käsin. Laske sama luku käsin tietokannasta ja vertaa sitä reitin palauttamaan lukuun.

Valmis kun: Koostereitti palauttaa tilastot, ja luvut täsmäävät käsin ajettuun tarkistuskyselyyn.

Tallenna työnäyte: Reitti commitilla ja tarkistuskyselyn tuloste työviikon 8 päiväkirjaan.

Työvaihe 2 / 4 · Rakenna profiilisivu itse rautalangan mukaan

- ☐ Reititä nimen mukaan. Lisää reitti `/kayttaja/:nimi` react-routerin parametrilla ja lue nimi sivulla.
- ☐ Rakenna komponentit. Tee itse ProfiiliOtsikko, TilastoKortit, LaiteJakauma, PelaaNytLista ja HistoriaLista ilman valmista käyttöliittymäkirjastoa.
- ☐ Hae data omalla hookilla. Hae profiilin kooste omalla hookilla ja käsittele lataus- ja virhetila kuten työviikolla 4.
- ☐ Käsittele tuntematon nimi. Näytä selkeä viesti, jos käyttäjää ei löydy.
- ☐ Jäljitä virheet. Kun data ei näy odotetusti, jäljitä syy kehittäjätyökalujen Network-välilehdeeltä ja palvelinlokista.

Valmis kun: Profiilisivu aukeaa osoitteesta `/kayttaja/<nimi>`, ja tuntematon nimi näyttää selkeän viestin.

Tallenna työnäyte: Muutokset commitilla ja kuvakaappaus profiilisivusta `project-docs/-`-kansioon.

Työvaihe 3 / 4 · Tarkista profiili vierailijana

- ☐ Avaa yksityinen ikkuna. Avaa profiili yksityisessä selainikkunassa ilman kirjautumista.
- ☐ Tarkista näkyvät tiedot. Tarkista, että tilastot, pelaa nyt -lista ja historia näkyvät.
- ☐ Tarkista piilotetut toiminnot. Tarkista, ettei vierailija näe tilanappia eikä muokkaustoimintoja.
- ☐ Vertaa tietokantaan. Vertaa sivun tilastolukuja tietokannan tarkistuskyselyyn.

Valmis kun: Vierailija näkee profiilin tilastoineen yksityisessä ikkunassa, ja luvut täsmäävät tietokantaan.

Tallenna työnäyte: Kuvakaappaus vierailijanäkymästä `project-docs/-`-kansioon ja vertailun tulos työviikon 8 päiväkirjaan.

Työvaihe 4 / 4 · Vertaa profiilia rautalankaan ja aja testitapaus T09

- ☐ Aseta rinnakkain. Aseta työviikon 2 rautalanka ja profiilisivun kuvakaappaus rinnakkain samaan kuvaan tai dokumenttiin.
- ☐ Kirjaa poikkeamat. Kirjaa jokainen poikkeama rautalangasta ja sen perustelu.
- ☐ Kirjaa testitapaus T09. Raja: profiili, jolla ei ole yhtään peliä, näyttää nollatilastot ja tyhjän tilan viestin eikä kaadu. Kirjaa odotus ennen ajoa.
- ☐ Aja testitapaus T09. Aja testi uudella käyttäjällä ja kirjaa toteutunut tulos testimatriisiin.
- ☐ Tarkista virheenkorjausketju 1. Jos löysit tällä viikolla aidon bugin, kirjaa se virheenkorjausketjuksi 1 testimatriisiin. Jos et löytänyt, kirjaa se päiväkirjaan.

Valmis kun: Rautalanka ja toteutus ovat rinnakkain poikkeamineen, ja testitapaus T09 on ajettu.

Tallenna työnäyte: Rinnakkaiskuva `project-docs/-`-kansioon, T09 `project-docs/testimatriisi.md`-tiedostoon ja poikkeamat perusteluineen työviikon 8 päiväkirjaan.

Viikon lopputarkistus: Vierailija näkee profiilin tilastoineen yksityisessä selainikkunassa, ja tilastoluvut täsmäävät tietokannan tarkistuskyselyyn; rautalankavertailu on päiväkirjassa.

Työnäyte Git-repositoryyn ennen rastia: Kuvakaappaus ja rautalanka rinnakkain, commitit, tarkistuskyselyn tuloste, T09 sekä ketju 1:n kirjaus, jos aito bugi osui tälle viikolle.

Työviikko 9 / 18 – Haku ja suodatus

Yhteys kokonaisprojektiin: Työviikon 8 profiili näytti, että pelejä kertyy nopeasti. Nyt pelilista saa haun ja suodatuksen, jotka tehdään tietokantakyselyssä eikä selaimen muistissa. Samalla tuot toisen ulkoisen komponentin ja kirjaat kokoelmien päätöskriteerit, jotta asiakas voi työviikolla 10 kokeilla koko palvelua.

Viikon tavoite: Jäsen löytää pelin nimen, laitteen ja tilan mukaan, ja haku ilman osumia kertoo, miten pääsee takaisin koko listaan.

Työvaihe 1 / 4 · Toteuta haku ja suodatus parametrisoiduilla kyselyillä

- ☐ Lisää kyselyparametrit. Lisää reittiin GET /api/games parametrit nimi, laite ja tila.
- ☐ Kokoa ehdot listasta. Rakenna WHERE-ehdot listasta ja anna arvot aina parametreina. Älä liimaa arvoja kyselymerkkijonoon.
- ☐ Suodata tietokannassa. Tee suodatus kyselyssä. Älä suodata selaimessa koko dataa.
- ☐ Kokeile injektia. Syötä hakuun heittomerkki tai pätkä SQL:ää ja tarkista, että kysely käsittelee sen tekstinä.

Valmis kun: Suodatus toimii kyselyparametreilla suoraan rajapinnasta, eikä hakuun syötetty SQL muuta kyselyä.

Tallenna työnäyte: Muutokset commitilla ja esimerkikutsut tuloksineen työviikon 9 päiväkirjaan.

Työvaihe 2 / 4 · Rakenna suodatusnäkyvä ja tyhjän tuloksen tila

- ☐ Tee suodatuskentät. Tee hakukenttä nimelle ja valinnat laitteelle ja tilalle.
- ☐ Päivitä tulokset. Hae tulokset rajapinnasta aina, kun suodatin muuttuu.
- ☐ Tee tyhjä tila. Näytä viesti, kun hakuun ei löydy osumia, ja nappi, joka nollaa suodattimet.
- ☐ Vertaa suoraan kutsuun. Aja sama suodatus käyttöliittymästä ja suoraan osoitteesta kyselyparametreilla. Tulosten pitää olla samat.

Valmis kun: Suodatus "NES + pelattu läpi" palauttaa samat rivit käyttöliittymässä ja suorassa kutsussa, ja tyhjä haku näyttää viestin ja nollausnapin.

Tallenna työnäyte: Kuvakaappaus käyttöliittymän ja suoran kutsun rinnakkaisista tuloksista project-docs/-kansioon ja muutokset commitilla.

Työvaihe 3 / 4 · Valitse ja ota käyttöön toinen ulkoinen komponentti

- ☐ Nimeä tarve. Päättää, mihin tarpeeseen komponentti tulee, esimerkiksi kaavio profiiliin.
- ☐ Vertaile kaksi ehdokasta. Täytä vertailu: tarve, koko ja riippuvuudet, saavutettavuus ja arvio itse tekemisen työmäärästä.
- ☐ Asenna ja konfiguroi. Asenna valittu komponentti ja käytä sitä yhdessä paikassa. Kirjaa, mitä asetuksia muutit ja miksi.
- ☐ Kirjaa perustelu. Kirjoita perustelumuiot project-docs/-kansioon ja päätös suunnitelman kenttään Toinen ulkoinen komponentti.
- ☐ Kirjaa riippuvuus. Lisää README:hen kirjasto, versio ja mihin sitä käytetään.

Valmis kun: Komponentti toimii palvelussa, ja perustelumuiotissa ovat vertailu, valinta ja arvio itse tekemisen työmäärästä.

Tallenna työnäyte: Perustelumuiot project-docs/-kansioon, päätös project-docs/suunnitelma.md-tiedostoon ja asennus commitilla.

Työvaihe 4 / 4 · Kirjaa kokoelmien kriteerit ja aja testitapaukset T10 ja T11

- ☐ Kirjaa kokoelmien kriteerit. Kirjaa suunnitelman kenttään Kokoelmien laajennuspäätös, millä ehdoilla kokoelmat toteutetaan. Päätös tehdään vasta työviikolla 11.
- ☐ Kirjaa testitapaus T10. Normaali käyttö: suodatus laitteen ja tilan mukaan palauttaa vain oikeat rivit. Kirjaa odotus ennen ajoa.
- ☐ Kirjaa testitapaus T11. Raja: haku ilman osumia näyttää tyhjän tilan viestin. Kirjaa odotus ennen ajoa.
- ☐ Aja ja kirjaa. Aja testitapaukset ja kirjaa toteutuneet tulokset testimatriisiin.

Valmis kun: Kokoelmien päätöskriteerit ovat suunnitelmassa, ja testitapaukset T10 ja T11 on ajettu.

Tallenna työnäyte: project-docs/suunnitelma.md ja project-docs/testimatriisi.md commitilla. Kriteerit työviikon 9 päiväkirjaan.

Viikon lopputarkistus: Suodatus "NES + pelattu läpi" palauttaa vain oikeat rivit sekä käyttöliittymässä että suoralla API-kutsulla; haku ilman osumia näyttää tyhjän tilan viestin; perustelumuiot ulkoisesta komponentista ja kokoelmien päätöskriteerit ovat repositoryssa.

Työnäyte Git-repositoryyn ennen rastia: Commitit, perustelumuiot ulkoisesta komponentista, API-kutsun ja käyttöliittymän rinnakkaiset tulokset, T10–T11 sekä laajennuspäätöksen kriteerikirjaus.

Työviikko 10 / 18 – Asiakaskatselmointi

Yhteys kokonaisprojektiin: Vaiheet 2 ja 3 on rakennettu omien oletusten varassa. Nyt asiakkaan edustaja käyttää julkaistua palvelua itse koko polun rekisteröitymisestä toisen jäsenen profiiliin. Hänen havainnoistaan johdetaan vaiheen 4 työjärjestys ja työviikon 11 palautemuutos.

Viikon tavoite: Asiakkaan edustaja on kokeillut julkaistua palvelua itse, ja hänen havaintonsa ovat priorisoituina issueina.

Työvaihe 1 / 4 · Valmistele katselmointirunko ja demopolku

- ☐ Varmista katselmoija. Tarkista ohjaajalta asiakkaan edustaja. Hän on nimetty ulkopuolinen henkilö, ei oma ohjaava opettajasi.
- ☐ Kirjoita demopolku. Kirjaa katselmointimuistioon, mitä asiakas tekee itse: rekisteröityy, lisää pelin, vaihtaa tilan, katsoo profiiliaan ja löytää toisen jäsenen profiilin.
- ☐ Kirjoita loppukysymykset. Kirjaa kysymykset, jotka esität lopuksi, esimerkiksi mikä oli helpointa ja mitä puuttuu, jotta asiakas ottaisi palvelun käyttöön.
- ☐ Tarkista versio. Tarkista, että demopolku toimii julkaistussa versiossa, ja kirjaa version commit muistioon.

Valmis kun: Katselmointimuistiossa ovat demopolku, loppukysymykset ja julkaistun version tunniste.

Tallenna työnäyte: Katselmointimuistion runko `project-docs/-kansioon` commitilla.

Työvaihe 2 / 4 · Pidä katselmointi, jossa asiakas käyttää palvelua itse

- ☐ Kirjaa rooli ja ajankohta. Kirjaa muistioon katselmoijan rooli ja ajankohta, ei nimeä. Nimet eivät kuulu julkiseen repositoryyn.
- ☐ Anna asiakkaan käyttää. Anna asiakkaan tehdä demopolku itse. Älä neuvo, ellei työ pysähdy kokonaan.
- ☐ Kirjaa omat sanat. Kirjaa lyhyinä lainauksina, mitä asiakas tekee ja sanoo, vähintään viisi havaintoa. Tarkemmat muistiinpanot lähetät ohjaajalle Teamsissa, jos hän pyytää.
- ☐ Esitä loppukysymykset. Esitä valmistellut kysymykset ja kirjaa vastaukset asiakkaan sanoin.

Valmis kun: Muistiossa ovat rooli, ajankohta ja vähintään viisi asiakkaan havaintoa hänen omilla sanoillaan.

Tallenna työnäyte: Katselmointimuistio `project-docs/-kansioon` commitilla.

Työvaihe 3 / 4 · Erotta havainnot tulkinnasta ja priorisoi palaute yhdessä

- ☐ Erotta sarakkeet. Kirjaa muistioon asiakkaan sanat ja oma tulkintasi eri sarakkeisiin.
- ☐ Tarkista erottelu. Lue muistio läpi ja tarkista, että jokaisesta rivistä näkee, mikä on asiakkaan sanomaa.
- ☐ Priorisoi yhdessä. Päättäkää yhdessä ohjaajan ja asiakkaan edustajan kanssa, mitkä muutokset tehdään ennen v1.0:aa ja mitkä siirtyvät v1.1:een.
- ☐ Kirjaa päätökset. Merkitse jokaiselle havainnolle päätös: ennen v1.0:aa tai v1.1.

Valmis kun: Jokaisella havainnolla on erillinen tulkinta ja yhdessä sovittu päätös.

Tallenna työnäyte: Päivitetty katselmointimuistio commitilla. Yllättävin havainto työviikon 10 päiväkirjaan.

Työvaihe 4 / 4 · Päivitä issuet ja lähetä asiakkaalle yhteenveto

- ☐ Tee issueet. Tee jokaisesta ennen v1.0:aa sovitusta muutoksesta oma issue, joka viittaa muistion havaintoon.
- ☐ Järjestä issue-tila. Järjestä issue-tila uudelleen ja merkitse v1.1:een siirtyvät erikseen.
- ☐ Kirjoita yhteenveto. Kirjoita asiakkaalle lyhyt viesti ilman teknistä jargonia: mitä muutetaan ja mitä siirtyy.
- ☐ Lähetä ja tallenna. Lähetä viesti ja tallenna se tiedostoon `project-docs/viestit.md`. Kirjaa vastaanottajasta vain rooli.

Valmis kun: Jokaisesta sovitusta muutoksesta on issue, ja yhteenveto on lähetetty ja tallennettu.

Tallenna työnäyte: Issue-tilun linkki työviikon 10 päiväkirjaan ja `project-docs/viestit.md` commitilla.

Viikon lopputarkistus: Katselmointimuistiossa on vähintään viisi asiakkaan havaintoa hänen omilla sanoillaan erillään omasta tulkinnasta, ja issueissa on niistä johdetut priorisoidut tehtävät.

Työnäyte Git-repositoryyn ennen rastia: Katselmointimuistio (rooli, ajankohta ja lainaukset), päivitetty issue-tila ja yhteenvetoviesti.

Vaihe 4 – Palaute, laatu ja käyttöönotto-ohje

Työviikko 11 / 18 – Palautemuutos

Yhteys kokonaisprojektiin: Työviikon 10 palaute muuttuu nyt koodiksi. Teet tärkeimmän muutoksen omassa haarassa ja yhdistät sen pääversioon pull requestilla, ja samalla suunnitelman arviot testataan toteumaa vasten. Kokoelmapäätös ratkaisee, mitä vaiheen 4 loppuviikoilla rakennetaan.

Viikon tavoite: Asiakkaan tärkein toive on muutettu palveluun, ja muutos on tuotannossa pull requestin kautta.

Työvaihe 1 / 4 · Valitse palautteen tärkein muutos ja kirjaa kriteerit

- ☐ Valitse muutos. Valitse katselmointimuistiosta tärkein ennen v1.0:aa sovittu muutos.
- ☐ Viittaa havaintoon. Kirjaa issueen, mihin muistion havaintoon muutos perustuu. Lainaa asiakkaan sanat.
- ☐ Kirjaa kriteerit. Kirjoita issueen havaittavat hyväksymiskriteerit ennen koodia.
- ☐ Arvioi työmäärä. Kirjaa issueen työmääräarvio, jota verrataan toteumaan tämän viikon lopussa.

Valmis kun: Issueessa ovat viittaus asiakkaan havaintoon, hyväksymiskriteerit ja työmääräarvio.

Tallenna työnäyte: Issue GitHubissa. Valinnan perustelu työviikon 11 päiväkirjaan.

Työvaihe 2 / 4 · Toteuta muutos omassa haarassa pienin commitein

- ☐ Luo haara. Luo haara muodossa feature/..., esimerkiksi feature/tilanappi-ensisijaiseksi.
- ☐ Tee pienet commitit. Tee muutos pienin commitein: yksi looginen muutos kerrallaan ja kuvaava viesti.
- ☐ Testaa kriteereitä vasten. Tarkista jokainen hyväksymiskriteeri ja aja siihen liittyvä testitapaus uudelleen.
- ☐ Lisää uusi testitapaus. Kirjaa muutokselle testimatriisiin uusi testitapaus odotuksineen ennen ajoa ja aja se.

Valmis kun: Muutos on omassa haarassa kuvaavasti nimetyin commitein, ja hyväksymiskriteerit on tarkistettu.

Tallenna työnäyte: Haara ja commitit repositoryssa. Testitulokset project-docs/testimatriisi.md-tiedostoon.

Työvaihe 3 / 4 · Yhdistä muutos pääversioon pull requestilla

- ☐ Avaa pull request. Avaa pull request ja kirjoita kuvaus: mitä, miksi, miten testattu sekä riskit ja rajaukset.
- ☐ Katselmoi oma muutos. Käy muutos läpi kuin vieraan koodina ja kirjoita pull requestiin katselmointikommentti.
- ☐ Yhdistä ja ratkaise konflikti. Yhdistä pääversioon ja ratkaise konflikti hallitusti. Jos konfliktia ei synny, pyydä ohjaajalta harjoituskonflikti.
- ☐ Julkaise ja tarkista. Julkaise pääversio ja tarkista muutos tuotannossa.

Valmis kun: Pull requestissa ovat kuvaus, viittaus palautteeseen ja katselmointikommentti, ja muutos on tuotannossa.

Tallenna työnäyte: Pull requestin linkki työviikon 11 päiväkirjaan.

Työvaihe 4 / 4 · Päivitä suunnitelma ja tee kokoelmapäätös

- ☐ Vertaa arviot toteumaan. Kirjaa suunnitelmaan issueiden arvio ja toteuma tunteina sekä se, mitä opit arvioinnista.
- ☐ Tee kokoelmapäätös. Päätä työviikon 9 kriteereillä: kokoelmat omana haarana vaiheessa 4 tai v1.1-listalle. Siirto on rajauspäätös, ei epäonnistuminen.
- ☐ Lataa suunnitelma. Päivitä suunnitelmalomakkeen kentät, lataa suunnitelma.md ja korvaa repositoryn vanha tiedosto.
- ☐ Kerro asiakkaalle. Kerro asiakkaalle lyhyesti ilman jargonia, mikä muuttui, ja tallenna viesti tiedostoon project-docs/viestit.md.

Valmis kun: Suunnitelman muutosvertailu (diff) näyttää päivitetty arviot ja kokoelmapäätöksen perusteluineen, ja asiakasviesti on tallennettu.

Tallenna työnäyte: project-docs/suunnitelma.md ja project-docs/viestit.md commitilla.

Viikon lopputarkistus: PR:ssä on kuvaus, viittaus palautteeseen ja katselmointikommentti; muutos on tuotannossa; suunnitelman muutosvertailu (diff) näyttää päivitetty arviot.

Työnäyte Git-repositoryyn ennen rastia: Pull requestin linkki ja commit-historia, suunnitelman diff kirjattuine kokoelmapäätöksineen sekä asiakasviesti.

Työviikko 12 / 18 – Responsiivisuus ja saavutettavuus

Yhteys kokonaisprojektiin: Kaikki näkymät ovat nyt olemassa, ja palautemuutos on tehty. Ennen testausviikkoa viimeistelet ne laitteille ja käyttötavoille, joita et itse käytä kehittäessäsi: puhelimelle ja pelkälle näppäimistölle. Työviikolla 13 koko palvelu testataan tässä kunnossa tuotantoa vasten.

Viikon tavoite: Jäsen lisää pelin puhelimella ja pelkällä näppäimistöllä, ja Lighthouse-luvut ennen ja jälkeen näyttävät, mikä parani.

Työvaihe 1 / 4 · Päättää taitekohdat ja korjaa näkymät puhelimelle

- ☐ Aja Lighthouse ennen. Aja Lighthouse keskeisille näkymille ja ota ennen-luvut talteen ennen ensimmäistäkään korjausta.
- ☐ Käy polut puhelimella. Käy rekisteröityminen, pelin lisäys, tilanvaihto ja profiili läpi oikealla puhelimella.
- ☐ Päättää taitekohdat. Päättää taitekohdat sen mukaan, missä sisältö hajoaa, ei laitemallien mukaan.
- ☐ Korjaa näkymät. Korjaa keskeiset näkymät omalla CSS:llä niin, ettei vaakavieritystä synny.
- ☐ Kirjaa perustelu. Kirjoita taitekohdat perusteluineen suunnitelman kenttään Responsiivisuuden taitekohdat.

Valmis kun: Keskeiset näkymät toimivat puhelimella ilman vaakavieritystä, ja taitekohdat on perusteltu suunnitelmassa.

Tallenna työnäyte: Lighthouse-raportit ennen ja kuvakaappaukset puhelimelta `project-docs/-`-kansioon, taitekohdat suunnitelmaan ja CSS commitilla.

Työvaihe 2 / 4 · Korjaa näppäinpolku ja fokus

- ☐ Irrota hiiri. Tee rekisteröityminen ja pelin lisäys alusta loppuun pelkällä näppäimistöllä.
- ☐ Tarkista fokusjärjestys. Tarkista, että fokus etenee sivun luettavassa järjestyksessä ylhäältä alas.
- ☐ Tarkista fokuksen näkyvyys. Tarkista, että fokus näkyy jokaisessa kohteessa, myös omissa napeissa.
- ☐ Korjaa katkot. Korjaa jokainen kohta, johon jäit jumiin, ja kirjaa korjaus omin sanoin.

Valmis kun: Rekisteröityminen ja pelin lisäys onnistuvat pelkällä näppäimistöllä, ja fokus näkyy koko ajan.

Tallenna työnäyte: Korjaukset commitilla ja näppäintestauksen löydökset työviikon 12 päiväkirjaan.

Työvaihe 3 / 4 · Tarkista kontrastit, tekstivastineet ja otsikot

- ☐ Mittaa kontrastit. Mittaa tekstin kontrasti myös violetilla aksentilla. Vaatimus on vähintään 4.5:1.
- ☐ Tarkista alt-tekstit. Anna jokaiselle merkitykselliselle kuvalle alt-teksti, esimerkiksi profiilikuvalle.
- ☐ Tarkista label-kytkennät. Tarkista, että jokainen kenttä ja virheviesti on kytketty ohjelmallisesti.
- ☐ Tarkista otsikot. Tarkista, että otsikot etenevät $h1 \rightarrow h2 \rightarrow h3$ ilman hyppyjä.

Valmis kun: Kontrastit täyttävät vaatimuksen, ja jokaisella kentällä ja merkityksellisellä kuvalla on tekstivastine.

Tallenna työnäyte: Korjaukset commitilla ja tarkistuslistan tulos työviikon 12 päiväkirjaan.

Työvaihe 4 / 4 · Aja Lighthouse uudelleen ja testitapaus T12

- ☐ Aja Lighthouse jälkeen. Aja Lighthouse samoille näkymille kuin ennen korjauksia ja tallenna raportit.
- ☐ Selitä erot. Selitä omin sanoin, mistä korjauksesta kukin muutos tuli.
- ☐ Kirjaa testitapaus T12. Raja: profiilisivu 320 pikselin leveydellä ilman vaakavieritystä, ja toiminnot ovat käytettävissä. Kirjaa odotus ennen ajoa.
- ☐ Aja testitapaus T12. Aja testi ja kirjaa toteutunut tulos testimatriisiin.

Valmis kun: Lighthouse-raportit ennen ja jälkeen on tallennettu, jokainen ero on selitetty, ja testitapaus T12 on ajettu.

Tallenna työnäyte: Lighthouse-raportit `project-docs/-`-kansioon, T12 `project-docs/testimatriisi.md`-tiedostoon ja luvut työviikon 12 päiväkirjaan.

Viikon lopputarkistus: Pelin lisäys onnistuu puhelimella ja pelkällä näppäimistöllä; Lighthouse-saavutettavuus on kirjattu ennen ja jälkeen, ja jokainen korjaus on selitetty omin sanoin.

Työnäyte Git-repositoryyn ennen rastia: Kuvakaappaukset laitteelta, Lighthouse-raportit ennen ja jälkeen, commitit sekä T12.

Työviikko 13 / 18 – Testausviikko

Yhteys kokonaisprojektiin: Työviikkojen 4–12 aikana testimatriisiin kertyi yksittäisiä testitapauksia. Nyt matriisi täydennetään ja ajetaan ensimmäistä kertaa kokonaisuutena tuotantoa vasten, ja löydökset korjataan täydellisinä ketjuina. Matriisi on turvaverkko, jota vasten koodia siistitään työviikolla 14.

Viikon tavoite: Koko testimatriisi on ajettu tuotantoversiota vasten, ja löydökset on korjattu täydellisinä ketjuina regressiotestiin asti.

Työvaihe 1 / 4 · Täydennä testimatriisi kolmeen luokkaan

- Kokoa aiemmat tapaukset. Tarkista, että testimatriisissa ovat kaikki aiemmat testitapaukset T01–T12.
- Lisää uudet tapaukset. Lisää tapauksia niin, että vähintään 12 on kirjoitettu ja ajettu työviikkoon 13 mennessä kolmessa luokassa: normaali käyttö, rajat ja virhetilanteet.
- Tasapainota luokat. Tarkista, että jokaisessa luokassa on suunnilleen kolmannes tapauksista.
- Kirjaa odotukset. Kirjoita jokaisen tapauksen odotettu tulos ennen kuin ajat sen.

Valmis kun: Matriisissa on vähintään 12 testitapausta kolmessa luokassa, ja jokaisella on odotettu tulos ennen ajoa. Koko projektissa tapauksia on vähintään 14, sillä testitapaus T14 ajetaan työviikolla 16.

Tallenna työnäyte: `project-docs/testimatriisi.md` commitilla.

Työvaihe 2 / 4 · Aja koko matriisi tuotantoversiota vasten

- Varmista versio. Tarkista, että julkaistu versio on sama kuin pääversio, ja kirjaa sen commit matriisiin.
- Aja tuotantoa vasten. Aja jokainen tapaus julkisessa osoitteessa. Älä oikaise omalla koneella.
- Kirjaa toteutunut tulos. Kirjaa jokaiselle riville toteutunut tulos heti ajon jälkeen.
- Kirjaa löydökset. Kirjaa jokainen epäonnistunut tapaus toistamisohjeineen niin tarkasti, että toinen henkilö pystyy toistamaan vian.

Valmis kun: Matriisin jokaisella rivillä on odotettu ja toteutunut tulos tuotantoversiosta.

Tallenna työnäyte: `project-docs/testimatriisi.md` commitilla ja epäonnistuneiden tapausten määrä työviikon 13 päiväkirjaan.

Työvaihe 3 / 4 · Korjaa löydökset virheenkorjausketjuina

- Jäljitä syy. Jäljitä syy kehittäjätyökaluilla, verkkopyynnöistä ja palvelinlokista.
- Korjaa ja uusintatestaa. Tee korjauscommit ja aja sama testitapaus uudelleen.
- Aja regressio. Aja ydinpolut uudelleen ja varmista, ettei korjaus rikkonut muuta.
- Kirjaa ketju 2. Kirjaa ketju kokonaan: havainto, toistamisohje, syy, korjauscommit, uusintatesti ja regressiotesti.
- Varmista, että ketju on aito. Keksittyä bugia ei kirjata. Jos aitoja löydöksiä ei ole, pyydä ohjaajalta vicatehtävä.

Valmis kun: Vähintään yksi täydellinen virheenkorjausketju on viety regressiotestiin asti tällä viikolla.

Tallenna työnäyte: Ketju 2 `project-docs/testimatriisi.md`-tiedostoon ja korjauscommitit repositoryyn.

Työvaihe 4 / 4 · Aja testitapaus T13 katkenneella verkolla

- Kirjaa testitapaus T13. Virhetilanne: verkkoyhteys katkeaa kesken lomakkeen lähetyksen. Kirjaa odotettu tulos ennen ajoa.
- Katkaise yhteys. Katkaise yhteys kehittäjätyökalujen Network-välilehdellä (Offline) juuri ennen lähetystä.
- Kirjaa tulos. Kirjaa, mitä käyttäjä näki ja jäikö tietokantaan puolikas rivi.
- Tarkista poikkeama. Jos tulos poikkesi odotuksesta, korjaa se samalla ketjumallilla kuin työvaiheessa 3.

Valmis kun: Testitapaus T13 on ajettu ja kirjattu, ja mahdollinen poikkeama on korjattu ketjuna.

Tallenna työnäyte: `project-docs/testimatriisi.md` commitilla.

Viikon lopputarkistus: Vähintään 12 testitapausta on kirjoitettu ja ajettu työviikkoon 13 mennessä: matriisin jokaisella rivillä on odotettu tulos (kirjattu ennen ajoa) ja toteutunut tulos. Vähintään yksi täydellinen ketju on viety regressiotestiin asti tällä viikolla.

Työnäyte Git-repositoryyn ennen rastia: Testimatriisi tuloksineen, ketjukirjaukset ja korjauscommitit.

Työviikko 14 / 18 – Laatuviikko

Yhteys kokonaisprojektiin: Työviikon 13 testimatriisi antaa turvaverkon. Nyt koodia voi siistiä ja todistaa regressioajolla, ettei toiminta muuttunut, ja samalla arvioit tietoturvan koko palvelulle. Siisti ja arvioitu koodi on pohja, jolle työviikon 15 käyttöönotto-ohje kirjoitetaan.

Viikon tavoite: Koodin rakenne on siistitty pienin commitein ilman toiminnan muutosta, ja tietoturva-arvio kertoo jokaisen riskin ratkaisun omassa koodissa.

Työvaihe 1 / 4 · Arvioi koodin rakenne ja listaa refaktorointikohteet

- ☐ Aja ydinrivit ennen. Aja testimatriisin ydinrivit ja kirjaa tulokset regressiokirjaukseen ennen ensimmäistäkään muutosta.
- ☐ Lue koodi vieraan silmin. Käy komponentit ja reittitiedostot läpi ja merkitse kohdat, joita et ymmärrä heti.
- ☐ Tunnista kohteet. Etsi komponentit, joilla on monta vastuuta, toistuva logiikka, epäselvät nimet ja reittitiedostot, jotka pitäisi jakaa moduuleihin.
- ☐ Kirjaa lista. Kirjaa refaktorointikohteet listaksi: kohde, ongelma ja aiottu muutos.

Valmis kun: Refaktorointikohteet on listattu, ja testimatriisin ydinrivien tulokset on kirjattu ennen muutoksia.

Tallenna työnäyte: Kohdelista ja regressiokirjauksen ennen-sarake `project-docs/-`-kansioon commitilla.

Työvaihe 2 / 4 · Refaktoroi lista pienin commitein

- ☐ Tee yksi kohde kerrallaan. Tee jokainen listan kohde omana committinaan.
- ☐ Nimeä commit kuvaavasti. Kirjoita viesti, joka kertoo muutoksen, esimerkiksi "PeliKortin tilanvaihto omaan hookkiin useTilanvaihto".
- ☐ Muuta vain rakennetta. Muuta vain rakennetta ja nimiä. Jos huomaat bugin, kirjaa se erikseen testimatriisiin.
- ☐ Kirjaa ennen ja jälkeen. Kirjaa listaan jokaisesta kohteesta, mitä muuttui, esimerkiksi `data2` → `profiiliKooste`.

Valmis kun: Jokainen listan kohde on omana kuvaavasti nimettynä committinaan.

Tallenna työnäyte: Refaktorointicommitit repositoryssa ja ennen ja jälkeen -muutokset kohdelistaan.

Työvaihe 3 / 4 · Kirjoita tietoturva-arvio omaa koodia vasten

- ☐ Käy riskit läpi. Käy tietoturvatarkeistuslista läpi omaa koodia vasten: syötteet, salasanat, istunnot tai tokenit, tietojen näkyvyys, SQL-injektio ja XSS.
- ☐ Kokeile XSS-hyökkäystä. Kirjoita pelin kommenttiin `<script>`-tagi ja tarkista, että sivu näyttää sen tekstinä eikä aja sitä.
- ☐ Kirjoita arvio. Kirjoita `project-docs/tietoturva-arvio.md`: riski, ratkaisu tässä projektissa ja missä koodissa. Vähintään viisi riskiä.
- ☐ Kirjaa avoimet riskit. Kirjaa loppuun rehellisesti riskit, joita et ehtinyt käsitellä.

Valmis kun: `project-docs/tietoturva-arvio.md` kattaa vähintään viisi riskiä ratkaisuihin ja koodiviittauksiin.

Tallenna työnäyte: `project-docs/tietoturva-arvio.md` commitilla. Avoimet riskit työviikon 14 päiväkirjaan.

Työvaihe 4 / 4 · Aja testimatriisin ydinrivit refaktoroinnin jälkeen

- ☐ Aja ydinrivit jälkeen. Aja samat ydinrivit kuin työvaiheessa 1, nyt refaktoroinnin jälkeen.
- ☐ Vertaa tuloksia. Kirjaa tulokset regressiokirjauksen jälkeen-sarakkeeseen ja vertaa niitä ennen-sarakkeeseen.
- ☐ Tarkista muutokset. Jos jokin tulos muuttui, peru tai korjaa kyseinen commit ja aja rivi uudelleen.

Valmis kun: Testimatriisin ydinrivit menevät läpi sekä ennen että jälkeen refaktoroinnin.

Tallenna työnäyte: Regressiokirjaus `project-docs/-`-kansioon commitilla.

Viikon lopputarkistus: Refaktorointicommitit ovat pieniä ja kuvaavasti nimettyjä; testimatriisin ydinrivit menevät läpi ennen ja jälkeen; `project-docs/tietoturva-arvio.md` kattaa vähintään viisi riskiä ratkaisuihin ja koodiviittauksiin.

Työnäyte Git-repositoryyn ennen rastia: Commit-sarja, `project-docs/tietoturva-arvio.md` ja regressiokirjaus.

Työviikko 15 / 18 – Dokumentaatio

Yhteys kokonaisprojektiin: Koodi on työviikon 14 jäljiltä siistiä ja testattua. Nyt siitä kirjoitetaan ohje, jolla yhdistys saa palvelun käyttöön ilman tekijää, ja todistaa sen itse kuivaharjoituksella puhtaassa kansiossa. Työviikolla 16 ulkopuolinen testaaja seuraa samaa ohjetta ilman suullista apua.

Viikon tavoite: Kuka tahansa saa palvelun käyntiin puhtaassa kansiossa pelkän README:n ja käyttöönotto-ohjeen avulla.

Työvaihe 1 / 4 · Kirjoita README ja käyttöönnotto-ohje

- ☐ Rakenna runko. Kirjoita README:n runko: mikä PeliHylly on ja kenelle, vaatimukset, asennus, ympäristömuuttujat, migraatiot, käynnistys, julkaisu ja tunnetut puutteet.
- ☐ Lisää odotetut tulosteet. Kirjoita jokaiseen askeleeseen odotettu tuloste tai näkyvä lopputulos, esimerkiksi "taulut luotu (5)".
- ☐ Tee `.env.example`. Tee `.env.example`, jossa on jokainen tarvittava muuttuja ilman todellisia salaisuuksia.
- ☐ Kirjaa v1.1-lista. Lisää loppuun tunnetut puutteet ja v1.1-lista.

Valmis kun: README kertoo, mitä palvelu tekee ja kenelle, ja jokaisella asennusaskeleella on odotettu tulos.

Tallenna työnäyte: README.md ja `.env.example` commitilla repositoryyn.

Työvaihe 2 / 4 · Dokumentoi riippuvuudet ja arkkitehtuuri

- ☐ Taulukoi riippuvuudet. Kirjoita taulukko: kirjasto, versio ja mihin sitä käytetään. Mukaan tulevat react-router ja toinen ulkoinen komponentti.
- ☐ Piirrä arkkitehtuuri. Piirrä yksi kuva client-server-tietokanta-rakenteesta ja lisää se README:hen.
- ☐ Kuvaa komponenttirakenne. Kuvaa pääkomponentit ja niiden vastuut lyhyesti.
- ☐ Kuvaa ympäristöasetukset. Kerro, mitkä asetukset eroavat kehityksessä ja tuotannossa ja mistä ne luetaan.

Valmis kun: README:ssä ovat riippuvuustaulukko, arkkitehtuurikuva, komponenttirakenne ja ympäristöasetukset.

Tallenna työnäyte: Päivitetty README.md ja arkkitehtuurikuva commitilla.

Työvaihe 3 / 4 · Tee kuivaharjoitus puhtaassa kansiossa

- ☐ Kloonaa puhtaaseen kansioon. Kloonaa repository uuteen kansioon, jossa ei ole `node_modules`-kansiota eikä `.env`-tiedostoa.
- ☐ Seuraa ohjetta kirjaimellisesti. Tee vain se, mitä ohjeessa lukee. Älä käytä muistiasi.
- ☐ Kirjaa tökkäykset. Kirjaa kuivaharjoituksen lokiin jokainen kohta, jossa jouduit arvaamaan: askel, ohjeen teksti, mitä tapahtui ja korjaus.
- ☐ Korjaa ohje. Korjaa jokainen tökkäys ohjeeseen ja aja korjattu askel uudelleen.

Valmis kun: `git clone` puhtaaseen kansioon ja ohjeen komennot tuottavat toimivan sovelluksen ilman yhtään ohjeen ulkopuolista temppua.

Tallenna työnäyte: Kuivaharjoituksen loki `project-docs/`-kansioon ja ohjeen korjaukset commitilla.

Työvaihe 4 / 4 · Kokoa julkaisuehdokkaan jäädytyslista

- ☐ Kokoa lista. Kokoa jäädytyksen tarkistuslista: ei uusia ominaisuuksia, ydinrivit menevät läpi, `.env.example` vastaa tuotantoa ja migraatiot toimivat puhtaaseen kantaan.
- ☐ Sovi versiomerkin nimen. Sovi julkaisuehdokkaan versiomerkin nimen, esimerkiksi `v1.0-rc1`.
- ☐ Käy lista läpi. Käy lista läpi nykyistä versiota vasten ja merkitse, mikä on vielä kesken.

Valmis kun: Jäädytyslista on repositoryssa, ja jokaisen kohdan tila on merkitty.

Tallenna työnäyte: Jäädytyslista `project-docs/`-kansioon commitilla ja sen sisältö työviikon 15 päiväkirjaan.

Viikon lopputarkistus: `git clone` puhtaaseen kansioon ja ohjeen komennot tuottavat toimivan sovelluksen ilman yhtään ohjeen ulkopuolista temppua; jokainen matkalla kirjattu tökkäys on korjattu ohjeeseen.

Työnäyte Git-repositoryyn ennen rastia: README ja `project-docs/`, `.env.example` sekä kuivaharjoituksen loki.

Vaihe 5 – Julkaisu ja näyttö

Työviikko 16 / 18 – Julkaisuehdokas ja julkaisutestaus

Yhteys kokonaisprojektiin: Työviikon 15 ohje pannaan nyt koetukselle. Jäädytät sisällön, julkaiset julkaisuehdokkaan ja annat toisen ulkopuolisen henkilön ottaa sen käyttöön puhtaassa ympäristössä ilman, että saat auttaa. Estäviksi luokitellut löydökset korjataan työviikolla 17 ennen v1.0:aa.

Viikon tavoite: Ulkopuolinen testaaaja ottaa julkaisuehdokkaan käyttöön pelkän kirjallisen ohjeen avulla, ja jokainen epäröinti on kirjattu ja luokiteltu.

Työvaihe 1 / 4 · Jäädytä sisältö ja julkaise julkaisuehdokas

- ☐ Jäädytä sisältö. Käy työviikon 15 jäädytyslistalla läpi. Tästä eteenpäin ei lisätä uusia ominaisuuksia.
- ☐ Kokoa tuotantobuild. Aja Viten tuotantobuild ja tarkista, että se syntyy virheettää.
- ☐ Merkitse versio. Luo julkaisuehdokkaan versiomerkinä eli tag, esimerkiksi v1.0-rc1, ja vie se etärepositoryyn.
- ☐ Julkaise asiakkaan ympäristöön. Julkaise julkaisuehdokas asiakkaan ympäristöön ja tarkista julkinen osoite.
- ☐ Kirjaa jäädytys. Kirjaa jäädytys suunnitelmaan, lataa suunnitelma.md ja korvaa repositoryn vanha tiedosto.

Valmis kun: Julkaisuehdokkaan tag on repositoryssa, ja sama versio vastaa asiakkaan ympäristössä.

Tallenna työnäyte: Tag repositoryssa, julkinen osoite työviikon 16 päiväkirjaan ja päivitetty project-docs/suunnitelma.md commitilla.

Työvaihe 2 / 4 · Järjestä julkaisutesti ulkopuoliselle testaajalle

- ☐ Varmista testaaaja. Sovi ohjaajan kanssa julkaisutestaaaja. Hän on eri henkilö kuin työviikon 10 asiakkaan edustaja.
- ☐ Kirjaa rooli ja ajankohta. Kirjaa julkaisutestin pöytäkirjaan testaaajan rooli, ajankohta ja testattava versio. Älä kirjaa nimeä.
- ☐ Anna vain ohje. Anna testaajalle repositoryn osoite, kirjallinen käyttöönotto-ohje ja palvelun osoite. Älä anna suullisia vinkkejä tai valmista .env-tiedostoa.
- ☐ Kirjaa testitapaus T14. Normaali käyttö: puhtaan ympäristön käyttöönotto pelkän kirjallisen ohjeen avulla. Kirjaa odotettu tulos ennen testiä.

Valmis kun: Pöytäkirjassa ovat testaaajan rooli, ajankohta ja versio, ja testitapaus T14 on kirjattu odotuksineen.

Tallenna työnäyte: Julkaisutestin pöytäkirja project-docs/-kansioon ja T14 project-docs/testimatriisi.md-tiedostoon.

Työvaihe 3 / 4 · Kirjaa testaaajan havainnot hänen omilla sanoillaan

- ☐ Havainnoi auttamatta. Testaaaja puhuu ja tekee, sinä kirjaat. Älä auta, ellei työ pysähdy kokonaan.
- ☐ Kirjaa vaiheittain. Kirjaa pöytäkirjaan vaiheet: käyttöönotto ohjeesta, rekisteröityminen, pelin lisääminen ja tilan vaihto.
- ☐ Kirjaa epäröinnit. Kirjaa jokainen epäröintikohta testaaajan sanoin.
- ☐ Tee korjauslista. Muuta epäröintikohdat ohjeen korjauslistaksi ja kirjaa testitapauksen T14 toteutunut tulos.

Valmis kun: Testaaaja rekisteröityi, lisäsi pelin ja vaihtoi tilan ilman suullista apua, tai jokainen epäröintikohta on kirjattu hänen sanoinaan.

Tallenna työnäyte: Pöytäkirja ja korjauslista project-docs/-kansioon commitilla.

Työvaihe 4 / 4 · Luokittele löydökset ja aloita ketju 3

- ☐ Luokittele yhdessä. Luokittele löydökset ohjaajan kanssa. Estävä löydös estää pakollisen toiminnon, kadottaa dataa tai vuotaa tietoa. Muut siirtyvät v1.1-listalle.
- ☐ Kirjaa perustelut. Kirjaa jokaisen luokittelun peruste pöytäkirjaan.
- ☐ Aloita ketju 3. Aloita estävästä löydöksestä virheenkorjausketju 3: havainto, toistamisohje ja syy.
- ☐ Kirjaa löydökset suunnitelmaan. Kirjaa julkaisutestin löydökset suunnitelmaan ja lataa päivitetty suunnitelma.md.

Valmis kun: Jokainen löydös on luokiteltu perusteluineen, ja estävien vikojen ketjut on aloitettu.

Tallenna työnäyte: Luokittelu pöytäkirjaan, ketju 3 project-docs/testimatriisi.md-tiedostoon ja project-docs/suunnitelma.md commitilla.

Viikon lopputarkistus: Testaaaja rekisteröityi, lisäsi pelin ja vaihtoi tilan ilman suullista apua, tai jokainen epäröintikohta on kirjattu ja luokiteltu; RC-tag on repositoryssa ja estävien vikojen ketjut aloitettu.

Työnäyte Git-repositoryyn ennen rastia: Julkaisuehdokkaan tagi, julkaisutestin pöytäkirja (rooli, ajankohta ja testaaajan sanat), löydösten luokittelu ja ketju 3:n aloitus.

Työviikko 17 / 18 – Julkaisu v1.0

Yhteys kokonaisprojektiin: Työviikon 16 estävät löydökset korjataan, ja ketju 3 viedään regressiotestiin asti. Sen jälkeen sama osoite, joka avattiin työviikolla 3, saa version v1.0, ja asiakas saa tiedotteen omalla kielellään. Julkaistu versio on se työnäyte, jonka esittelet näytössä työviikolla 18.

Viikon tavoite: Asiakkaan osoitteessa on versio v1.0, joka toimii myös laitteella, jolla palvelua ei ole ennen käytetty.

Työvaihe 1 / 4 · Korjaa estävät virheet ja vie ketju 3 loppuun

- ☐ Korjaa vain estävät. Korjaa työviikolla 16 estäviksi luokitellut virheet. Muut odottavat v1.1-listalla.
- ☐ Uusintatestaa. Aja jokaisen korjatun löydöksen testi uudelleen.
- ☐ Aja regressio. Aja testimatriisin ydinpolut uudelleen.
- ☐ Kirjaa ketju 3 loppuun. Kirjaa ketjuun korjauscommit, uusintatesti ja regressiotesti.

Valmis kun: Estävät virheet on korjattu ja uusintatestattu, ja ketju 3 on kirjattu regressiotestiin asti.

Tallenna työnäyte: Korjauscommitit repositoryyn ja ketju 3 `project-docs/testimatriisi.md`-tiedostoon.

Työvaihe 2 / 4 · Julkaise v1.0 tagilla ja perusta näyttömatrissi

- ☐ Tarkista salaisuudet. Tarkista, ettei yksikään salaisuus ole repositoryssa ja että ympäristömuuttujat on asetettu alustalle.
- ☐ Kokoa tuotantobuild. Aja tuotantobuild ja tarkista, että se syntyy virheett.
- ☐ Aja migraatiot. Aja migraatiot tuotannon tietokantaan.
- ☐ Merkitse versio. Luo tag `v1.0` ja vie se etärepositoryyn.
- ☐ Julkaise samaan osoitteeseen. Julkaise samaan osoitteeseen, jonka asiakas on tuntenut työviikosta 3 asti.
- ☐ Perusta näyttömatrissi. Luo `project-docs/nayttomatriisi.md`: rivi jokaiselle 32 vaatimukselle, jossa ovat tunnus, työnäytteen linkki ja viikko. Lisää ensimmäiseksi linkiksi tag `v1.0`.

Valmis kun: Repositoryssa on tag `v1.0`, ja sama versio vastaa asiakkaan tuntemassa osoitteessa.

Tallenna työnäyte: Tag repositoryssa, perustettu `project-docs/nayttomatriisi.md` commitilla ja julkaisun vaiheet työviikon 17 päiväkirjaan.

Työvaihe 3 / 4 · Kirjoita julkaisutiedote asiakkaalle

- ☐ Kerro, mitä palvelu tekee. Kerro ilman jargonia, mitä jäsen voi tehdä: rekisteröityä, lisätä pelin, vaihtaa tilan ja katsoa kaverin profiilia.
- ☐ Kerro, mitä siirtyi. Kerro, mitä siirtyi v1.1:een ja miksi, esimerkiksi profiiliviestit.
- ☐ Liitä osoite ja ohje. Liitä tiedotteeseen palvelun osoite ja tieto siitä, mistä käyttöönotto-ohje löytyy.
- ☐ Lähetä ja tallenna. Lähetä tiedote asiakkaalle ja tallenna se tiedostoon `project-docs/viestit.md`.

Valmis kun: Julkaisutiedote on lähetetty, ja se kertoo asiakkaan kielellä, mitä palvelu tekee ja mitä siirtyi jatkoon.

Tallenna työnäyte: `project-docs/viestit.md` commitilla.

Työvaihe 4 / 4 · Savutestaa tuotanto kahdella laitteella

- ☐ Avaa yksityinen ikkuna. Avaa osoite yksityisessä selainikkunassa ja tee koko polku rekisteröitymisestä tilanvaihtoon.
- ☐ Kokeile toisella laitteella. Tee sama polku laitteella, jolla et ole koskaan kirjautunut palveluun.
- ☐ Kirjaa savutesti. Kirjaa savutestiin molemmat laitteet ja tulokset.

Valmis kun: Palvelu toimii koko polulta yksityisessä selainikkunassa ja toisella laitteella.

Tallenna työnäyte: Savutestikirjaus `project-docs/-kansioon` ja tulokset työviikon 17 päiväkirjaan.

Viikon lopputarkistus: Osoite toimii yksityisessä selainikkunassa ja toisella laitteella; repositoryssa on tag `v1.0`; julkaisutiedote kertoo asiakkaan kielellä, mitä palvelu tekee ja mitä siirtyi jatkoon.

Työnäyte Git-repositoryyn ennen rastia: Tagi `v1.0`, tuotannon osoite, julkaisutiedote ja savutestikirjaus.

Työviikko 18 / 18 – Näyttö

Yhteys kokonaisprojektiin: 17 työviikkoa on tuottanut committeja, muistioita ja testitapauksia. Viimeisellä viikolla et rakenna mitään uutta, vaan teet niistä löydettäviä: jokainen vaatimus saa suoran linkin työnäytteeseen. Harjoiteltu demo ja itsearvio kertovat arvioijalle, mitä teit ja miksi.

Viikon tavoite: Arvioija avaa näyttömatrissista minkä tahansa rivin ja löytää siihen liittyvän työnäytteen alle minuutissa.

Työvaihe 1 / 4 · Täsmälinkit näyttömatriisiin 32 vaatimusta

- ☐ Linkitä rivi kerrallaan. Täydennä `project-docs/nayttomatriisi.md` rivi kerrallaan: vaatimuksen tunnus, työnäytteen linkki (commit, issue, tiedosto, tagi tai kuva) ja viikko. Sama työnäyte saa esiintyä useilla riveillä.
- ☐ Avaa jokainen linkki. Avaa linkki ja tarkista, että se osoittaa juuri oikeaan työnäytteeseen.
- ☐ Merkitse aukot. Merkitse rivit, joille et löydä työnäytettä.
- ☐ Käy aukot ohjaajan kanssa. Käy merkityt rivit läpi ohjaajan kanssa ja sovi, mikä työnäyte niihin kelpaa.

Valmis kun: Jokaisella 32 vaatimuksella on suora linkki työnäytteeseen, ja aukot on käyty läpi ohjaajan kanssa.

Tallenna työnäyte: `project-docs/nayttomatriisi.md` commitilla. Vaikeimmin linkitettävät rivit työviikon 18 päiväkirjaan.

Työvaihe 2 / 4 · Harjoittele 8–10 minuutin demo ääneen ja kellon kanssa

- ☐ Kirjoita demorunko. Kirjoita runko: palvelu ja asiakas, palvelu toiminnassa, yksi tekninen ratkaisu, yksi korjattu bugi, Git-historia ja tekoälyn käyttö sekä v1.1.
- ☐ Valitse esimerkit. Valitse esiteltävä tekninen ratkaisu, esimerkiksi tilahistorian transaktio, ja yksi täydellinen virheenkorjausketju.
- ☐ Harjoittele kellon kanssa. Harjoittele demo ääneen kellon kanssa vähintään kahdesti ja kirjaa kesto.
- ☐ Tarkista kesto. Jos demo ylittää 10 minuuttia, lyhennä runkoa ja harjoittele uudelleen.

Valmis kun: Demo on harjoiteltu vähintään kahdesti, ja se pysyy 8–10 minuutissa.

Tallenna työnäyte: Demorunko `project-docs/-kansioon` ja harjoiteltu kesto työviikon 18 päiväkirjaan.

Työvaihe 3 / 4 · Kirjoita itsearvio konkreettisin esimerkein

- ☐ Vastaa kysymyksiin. Vastaa itsearvion kysymyksiin: mikä onnistui, missä tarvitsit apua, minkä päätöksen tekisit toisin, miten tekoäly vaikutti ja mitä voisit seuraavaan projektiin.
- ☐ Liitä esimerkit. Liitä jokaiseen vastaukseen konkreettinen esimerkki ja linkki työnäytteeseen.
- ☐ Tarkista AI-loki. Tarkista, että AI-loki on ajan tasalla. Se tulee mukaan päiväkirjan lataukseen.
- ☐ Vie repositoryyn. Lataa koko projektipäiväkirja AI-lokeineen ja vie se repositoryyn itsearvion kanssa.

Valmis kun: Itsearviossa on vastaus jokaiseen kysymykseen konkreettisin esimerkein, ja itsearvio ja AI-loki ovat repositoryssa.

Tallenna työnäyte: Itsearvio `project-docs/-kansioon` ja päiväkirja AI-lokeineen tiedostoon `project-docs/projektipaivakirja.md`.

Työvaihe 4 / 4 · Luovuta paketti ohjaajalle ja pyydä luovutuskuittaus

- ☐ Pyydä tarkistus. Pyydä toista henkilöä avaamaan matriisin linkit ja valitsemaan satunnainen rivi. Työnäytteen pitää löytyä alle minuutissa.
- ☐ Korjaa rikkiäiset linkit. Korjaa jokainen linkki, joka ei auennut.
- ☐ Tarkista paketti. Käy luovutuspaketin tarkistuslista läpi: matriisi, päiväkirja, AI-loki, tag v1.0, julkinen osoite, itsearvio ja demorunko.
- ☐ Luovuta ja pyydä kuittaus. Luovuta paketti ohjaajalle ja pyydä luovutuskuittaus.

Valmis kun: Ohjaaja on kuitannut paketin, ja satunnainen matriisin rivi johtaa oikeaan työnäytteeseen alle minuutissa.

Tallenna työnäyte: Luovutuskuittaus työviikon 18 päiväkirjaan.

Viikon lopputarkistus: Ohjaaja avaa matriisista satunnaisen rivin ja päätyy oikeaan työnäytteeseen alle minuutissa; demo pysyy harjoituksessa ajassa; itsearvio ja AI-loki ovat päiväkirjan mukana repositoryssa.

Työnäyte Git-repositoryyn ennen rastia: Täsmälinkitetty näyttömatriisi, itsearvio, demorunko ja luovutuskuittaus.

Viimeiset viisi päivää

Päivä 1 · Sisältöjäädytys – viimeinen hyväksytty versio

Päivä 2 · Aineisto: päiväkirja, testit, AI-loki ja linkit

Päivä 3 · Harjoittelu: 8–10 min demo ja itsearviointi

Päivä 4 · Puskuri: tarkistus toisen henkilön kanssa

Päivä 5 · Luovutus ohjaajalle

Sanasto

Projektin tunnukset ja ammattitermit siinä järjestyksessä, jossa ne tulevat vastaan. Jokainen on selitetty myös sivustolla ensimmäisen käytön kohdalla.

P0	<i>Pakollinen ydin.</i> Toiminnot, joiden on valmistuttava, tai palvelua ei ole: rekisteröityminen ja kirjautuminen, pelien lisäys ja muokkaus, tilanvaihto ja tilahistoria, julkinen profiili sekä julkaisu tuotantoon. P0 tehdään ensin, ennen P1:tä ja P2:ta.
P1	<i>Tärkeä jatko.</i> Toiminnot, jotka tehdään sitten kun P0 toimii: haku ja suodatus, tähtiarvostelut ja kommentit sekä profiilikuva. Kokoelmat ovat P1-laajennus, josta päätetään erikseen työviikolla 11.
P2	<i>Valinnainen lisä.</i> Toiminnot, jotka saavat jäädä kokonaan pois ilman että työ on kesken. Tässä projektissa P2 tarkoittaa viestejä toisten profileihin: asiakas sanoi itse, että ne voivat jäädä.
GitHub-issue	<i>Yksi toteutustehtävä GitHubissa.</i> Yksi rajattu muutos palveluun GitHubin tehtävälistalla, esimerkiksi "#23 Tilan vaihto kirjaa historiariivin". Issuessa ovat tavoite, hyväksymiskriteerit, työmääräarvio ja testin tulos. Issue on yleensä puolen tai yhden päivän työ, ja yksi sivun työvaihe voi sisältää useita issueita. Issue tehdään Työtapa-sivun kuudella askeleella.
commit	<i>Tallennettu muutos.</i> Yksi nimetty muutos versionhallinnassa. Commitin viesti kertoo, mitä muutit; commit-tunniste on linkki, jolla työnäyte löytyy myöhemmin näyttömatriisista.
repository	<i>Projektin koodivarasto.</i> Versionhallinnan säilytyspaikka, jossa ovat koodi, dokumentit ja koko muutoshistoria. Työnäyte on aina repositoryssa, ei tällä sivulla.
haara	<i>Branch.</i> Erillinen kehityslinja, jossa muutoksen voi tehdä rauhassa ilman että pääversio rikkoutuu. Työviikosta 6 alkaen isommat muutokset tehdään omassa haarassa.
PR	<i>Pull request.</i> Pyyntö yhdistää oman haaran muutokset pääversioon. Pull requestin kuvaus kertoo mitä muutit, miksi ja miten testasit, ja muutos käydään läpi ennen yhdistämistä.
API	<i>Rajapinta.</i> Osoiteisto, jonka kautta selain hakee ja tallentaa tietoa palvelimelta. Yksi osoite on API-reitti: esimerkiksi GET /api/games palauttaa pelilistan.
JSON vko 2	<i>Tiedon siirtomuoto.</i> Tekstimuoto, jossa tieto liikkuu selaimen ja palvelimen välillä. Samaa muotoa voi käyttää myös tiedostoon tallentamiseen, ja siksi JSON-tiedosto on yksi tietovarastovaihtoehto.
tuotantobuild vko 3	<i>Julkaisua varten koottu versio.</i> Kehitystiedostoista koottu paketti, joka viedään julkiseen osoitteeseen. Se ei ole sama kuin kehityspalvelimen versio, joten se testataan aina erikseen.
T01 vko 4	<i>Testitapausten tunnus.</i> T tarkoittaa testitapausta ja numero yksilöi sen: T01 on ensimmäinen testitapausta, T02 toinen. Odotettu tulos kirjataan aina ennen ajoa, ja sama tunnus seuraa testiä testimatriisiin asti.
SQL vko 8	<i>Tietokannan kyselykieli.</i> Kieli, jolla tietokannalta kysytään tietoa ja jolla sinne kirjoitetaan. SQLite ymmärtää SQL:ää: SELECT hakee rivejä ja GROUP BY laskee koosteita.
XSS vko 14	<i>Cross-site scripting.</i> Hyökkäys, jossa käyttäjän syöttämä teksti – esimerkiksi pelin kommentti – päätyy sivulle koodina ja ajautuu muiden selaimessa. Torjunta on syötteen käsittely tekstinä, ei koodina.
CRUD	<i>Luonti, luku, muokkaus ja poisto.</i> Neljä perustoimintoa, joita tietoon kohdistetaan: uuden lisääminen, lukeminen, muokkaaminen ja poistaminen. PeliHyllyssä ne täydentyvät työviikkojen 4–9 aikana.
REST	<i>Yleinen rajapinnan tyyli.</i> Tapa rakentaa API niin, että jokaisella tietokohteella on oma osoite ja pyynnön menetelmä kertoo mitä tehdään: GET hakee, POST lisää, PATCH muokkaa ja DELETE poistaa.
RC vko 15	<i>Release candidate, julkaisuehdokas.</i> Lähes valmis versio, joka julkaistaan testattavaksi ennen lopullista julkaisua. RC1 on ensimmäinen julkaisuehdokas; sen jälkeen korjataan vain estävät virheet.
tag vko 16	<i>Versiomerkitä.</i> Nimi, joka kiinnitetään tiettyyn commitiin, jotta juuri se versio löytyy myöhemmin: v1.0-rc1 on julkaisuehdokas ja v1.0 valmis julkaisu.

Näyttömatriisi – 32 osaamisvaatimusta

Rasti vasta, kun vaatimukselle on täsmällinen työnäyte: linkki, commit, kuva, testirivi tai muistio. Sama työnäyte voi kelvata useaan kohtaan.

Ohjelmointi

- **käyttää ohjelmointieditoria tai kehitysympäristöä** – VS Code ja Viten kehityspalvelin käytössä koko projektin ajan; työkalutodennusten tulosteet työviikolta 1 ja kehittäjätyökalujen käyttö ketjukirjauksissa.
- **etsii ja korjaa virheitä ohjelmakoodista** – Virheenkorjausketju 1 aidosta bugista (ankkuri työviikko 8): syy jäljitetty kehittäjätyökaluilla ja verkkopyynnöistä, korjauscommit ja uusintatesti. Ketjut 2 ja 3 työviikoilla 13 ja 16.
- **testaa ohjelman toimintoja** – Testimatriisi, jossa on vähintään 12 tapausta kolmessa luokassa, odotettu tulos kirjattuna ennen ajoa ja uusintatestit korjausten jälkeen (työviikko 13).
- **käyttää rakenteista ohjelmointia toteutuksissa** – Komponenttijako PeliLista ja PeliKortti, oma fetch-hookki ja Express-reittien moduulijako työviikolla 4; syvennys refaktoroinnissa työviikolla 14.
- **kirjoittaa ylläpidettävää ohjelmakoodia** – Refaktorointicommit-sarja työviikolla 14: nimeäminen, toiston poisto ja vastuiden selkeytys; toiminta todistettusti ennallaan regressioajolla.
- **tulkitsee suunnitelmia ja toteuttaa käyttöliittymän tai sen osia** – Profiilisivu toteutettu työviikon 2 rautalangan mukaan; kuvakaappaus ja luonnos rinnakkain ja poikkeamat perusteltu (myös lomake työviikolla 5 ja responsiivisuus työviikolla 12).
- **tulkitsee suunnitelmia ja toteuttaa ohjelmiston toimintoja** – Tilanvaihto ja tilahistoria toteutettu käyttäjätarinan ja hyväksymiskriteerien mukaan työviikolla 6; kriteerit ja toteutus linkitetty issueen.
- **sopii tehtävistä tiimin muiden jäsenten kanssa** – Tehtävistä sopiminen ohjaajan kanssa työviikosta 2 alkaen; issue-taulu pitää tilanteen näkyvänä koko projektin ajan.
- **etsii ratkaisuvaihtoehtoja ja ratkoo ongelmia yhdessä tiimin kanssa** – Vertailumuistio istunto vastaan token omilla kriteereillä työviikolla 7; päätös tehty ja kirjattu yhdessä ohjaajan kanssa.
- **arvioi ratkaisujen toimivuuden yhdessä tiimin kanssa** – Asiakaskatselmointi työviikolla 10: täyttääkö toteutus käyttäjätarinat ja mitä muutetaan ennen v1.0:aa; priorisointi tehdään yhdessä.
- **arvioi omaa toimintaa tiimin jäsenenä** – Kirjallinen itsearvio työviikolla 18: mikä onnistui, missä tarvitsi apua ja mitä tekisi toisin, konkreettisin esimerkein.

Ohjelmistokehittäjänä toimiminen

- **selvittää kehitystiimin kanssa asiakkaan tarpeet** – Toimeksiannon purku ja vähintään kahdeksan kysymyksen lista työviikolla 1; vastaukset kirjattu ja viety käyttäjätarinoiksi työviikolla 2.
- **viestii tekniset asiat asiakaslähtöisesti** – Tilannekatsaus asiakkaalle julkaistusta rungosta ilman jargonia työviikolla 3; jatkuu katselmoinnin yhteenvedossa (työviikko 10) ja julkaisutiedotteessa (työviikko 17).
- **osallistuu version katselmointiin** – Asiakaskatselmointi julkaistusta versiosta työviikolla 10: runko, asiakkaan omat sanat ja sovitut muutokset; toinen katselmointi on julkaisutestaus työviikolla 16.
- **asettaa kehitystiimin kanssa toteutettavat toiminnot tärkeysjärjestykseen** – Käyttäjätarinoiden P0-, P1- ja P2-priorisointi ohjaajan kanssa työviikolla 2; profiiliviestit tietoisesti P2:een.
- **jakaa kehitystiimin kanssa toteutettavat toiminnot tehtäviksi** – P0-tarinat pilkottu issueiksi työviikolla 2; issue-taulu työn ohjauksen välineenä koko projektin ajan.
- **suunnittelee ja arvioi kehitystiimin kanssa tehtävien toteuttamista** – Työmääräarviot issueissa työviikolla 2; arvio vastaan toteuma päivitetty suunnitelmaan työviikolla 11.
- **kehittää ohjelmiston toimintalogiikkaa** – Tilakone sallittuine siirtymineen ja historiakirjaus transaktiossa työviikolla 6; lisäksi omistajuustarkistukset (työviikko 7) ja suodatuslogiikka (työviikko 9).
- **valitsee ohjelmistoon sopivan tietovaraston** – Vertailu SQLite, JSON-tiedosto ja palvelintietokanta datan rakenteen, käyttötilanteen ja laajuuden perusteella; kirjattu perustelu suunnitelmassa työviikolla 2.
- **toteuttaa yhteyden tietovarastoon** – SQLite-yhteys Expressistä työviikolla 4: skeeman migraatio, siemendata sekä luku, lisäys ja muokkaus hallitusti; koko CRUD eli luonti, luku, muokkaus ja poisto täydentyy työviikoilla 5–9.
- **hyödyntää rajapintoja ja käsittelee tietoa** – Oma REST-tyylinen rajapinta työviikolla 5: POST validointineen ja virhevastauksineen sekä selainpuolen fetch-käsittely virhetiloineen; kyselyparametrit työviikolla 9.
- **arvioi ohjelmiston tietoturvaa** – Dokumentoitu tietoturva-arvio työviikolla 14: syötteet, salasanahashit, istunnot, tietojen näkyvyys, SQL-injektio ja XSS; riski, ratkaisu ja koodiviittaus (perusta työviikolla 7).

- **käyttää versionhallintaa** – Git koko projektin ajan: etärepository työviikosta 1, kuvaavat commitit ja feature-haarat työviikosta 6 alkaen.
- **liittää ohjelman osan olemassa olevaan versioon** – Palautemuutos feature-haarassa ja pull request pääversioon työviikolla 11: kuvaus, itsekatselmointi, merge ja konfliktin ratkaisu.
- **julkaisee ohjelman tuotantoympäristöön** – v1.0 työviikolla 17: tuotantobuild, ympäristöasetukset, julkaisu valittuun pilviympäristöön, tagi ja savutesti; ensimmäinen julkaisu jo työviikolla 3.

Ohjelmiston toteuttaminen ohjelmistokomponenttikirjastolla

- **ottaa käyttöön ja konfiguroi ohjelmistokomponenttikirjaston käyttöön soveltuvan kehittämisympäristön** – Vite- ja React-projektin luonti ja konfigurointi työviikolla 1; kehitys- ja tuotantoasetukset kuntoon työviikkoon 3 mennessä.
- **selvittää ohjelmistokomponenttikirjaston tarjoamat mahdollisuudet ja rajoitteet** – k2-muistio työviikolla 3: mitä React ratkaisee (komponentit, tila, renderöinti) ja mitä ei (reititys, palvelin, tietovarasto), ja mitä siitä seuraa tälle projektille.
- **käyttää ohjelmistokomponenttikirjaston tärkeimpiä toimintoja ja työkaluja** – Profiilisivu työviikolla 8: komponentit, propsit, hookit (useState, useEffect ja oma hookki), reititysparametrit ja kirjautuneen tila contextilla (työviikko 7).
- **tuo kehittämisympäristöön ulkoisia komponentteja** – react-router työviikolla 3 ja toinen ulkoinen komponentti perusteltuna työviikolla 9: vertailu, konfigurointi ja riippuvuuden kirjaus dokumentaatioon.
- **suunnittelee, toteuttaa ja testaa ohjelmiston ohjelmistokomponenttikirjastoa käyttäen** – Kokonaisketju: komponenttirakenne ja vastuut suunniteltu työviikolla 2, toteutettu työviikoilla 3–12 ja testattu matriisilla työviikolla 13.
- **julkaisee ohjelmiston asiakkaan ympäristöön** – Viten tuotantobuild julkaisuehdokkaana asiakkaan ympäristöön julkaisutestausta varten työviikolla 16; v1.0 samaan osoitteeseen työviikolla 17.
- **dokumentoi ohjelmiston sovitulla tavalla** – README ja käyttöönotto-ohje työviikolla 15: asennus, käynnistys, riippuvuudet ja ympäristöasetukset; todistettu kuivaharjoituksella ja työviikon 16 ulkopuolisella testillä.

Muista: sivuston rastit ja kentät tallentuvat vain selaimeen. Ne eivät siirry opettajalle eivätkä korvaa Gitissä olevaa työtä.